Universidad Mariano Gálvez de Guatemala

Ingeniería en Sistemas de Información y Ciencias de la Computación

Curso:

Aseguramiento de la calidad del software

220260900482

Catedrático:

NANCY GUADALUPE PORTILLO PINTO

Sistema de Control de Líneas, Estaciones y Buses del Transmetro

Entregable final: documento unificado del proyecto

Ernesto Leonardo Oliva Esquivel 1290-23-14523

Semestre 8

24-10-2026

Índice

Introducción **3**

Resumen ejecutivo **4**

1. Análisis de requerimientos **5**

2. Diseño del sistema **8**

3. Estudios de factibilidad y propuesta monetaria **17**

4. Stack tecnológico y métricas de calidad **24**

5. Implementación del sistema **34**

5.1 Arquitectura 35

5.2 Stack implementado frente al propuesto 35

5.3 Estructura del repositorio 36

5.4 Modelo de datos implementado 36

5.5 Reglas de negocio y su verificación 37

5.6 Seguridad 37

5.7 Tiempo real y mapa 37

5.8 Publicación 37

6. Plan de pruebas, criterios y carta de aceptación **38**

7. Resultados de la ejecución de pruebas **50**

7.1 Registro de ejecución 51

7.2 Resultado por caso de prueba 51

7.3 Métricas de calidad obtenidas 52

7.4 Registro de defectos 52

8. Evidencias del sistema en funcionamiento **54**

9. Conclusiones y enlaces del proyecto **65**

9. Conclusiones 66

Trabajo pendiente 66

10. Enlaces y acceso al proyecto 67

Anexo A. Script de la base de datos en PostgreSQL **68**

Anexo B. Consultas de los reportes **74**

Introducción

Este documento reúne en un solo archivo todos los entregables del proyecto **Sistema de Control de Líneas, Estaciones y Buses del Transmetro**, desarrollado para el curso Aseguramiento de la Calidad del Software. Integra el análisis de requerimientos, el diseño (modelo entidad-relación, diagramas UML y script de base de datos), los estudios de factibilidad y la propuesta monetaria, el stack tecnológico con sus métricas de calidad, el plan de pruebas con los criterios y la carta de aceptación, y agrega la descripción de la implementación, los resultados reales de la ejecución de las pruebas, las evidencias del sistema en funcionamiento y los enlaces al repositorio y a la aplicación publicada.

Las secciones 1, 2, 3, 4 y 6 son los documentos entregados durante el curso, incorporados sin cambios. Las secciones 5, 7, 8, 9 y 10 y los anexos se elaboraron para esta entrega final a partir del sistema construido.

Objetivo general

Diseñar, construir y verificar un sistema web que unifique la administración y el monitoreo del Transmetro entre varias municipalidades, haciendo cumplir las reglas de operación y ofreciendo información y servicios al ciudadano, con un proceso de aseguramiento de la calidad documentado.

Objetivos específicos

- Registrar líneas, estaciones, accesos, guardias, buses, parqueos y pilotos, con las reglas de negocio validadas siempre en el servidor.
- Monitorear en tiempo real la posición de los buses y generar alertas automáticas de ocupación.
- Soportar el pago con tarjeta de acceso de forma segura, atómica e idempotente.
- Generar reportes exportables y tableros de control para la toma de decisiones.
- Verificar el cumplimiento de los requerimientos con un plan de pruebas trazable y pruebas automatizadas.

Enlaces del proyecto

Recurso	Enlace
Repositorio del código fuente, scripts de base de datos y documentación	https://github.com/ernesto615/Transmetro
Aplicación web publicada en la nube	https://transmetro-glvg.onrender.com
Presentación con el resumen ejecutivo	Archivo **Resumen_Ejecutivo_Transmetro.pptx** (adjunto a esta entrega)
Video de explicación general	[Agregar aquí el enlace del video]

La sección 10 detalla el contenido del repositorio, las cuentas de demostración y cómo ejecutar el proyecto.

Resumen ejecutivo

Problema. La operación del Transmetro involucra a varias municipalidades (Guatemala, Mixco, Villa Nueva, San José Pinula y Santa Catarina Pinula) que necesitan administrar de forma conjunta líneas, estaciones, buses, pilotos y recaudo, monitorear la flota y la ocupación en tiempo real y atender al ciudadano.

Solución. Una aplicación web en la nube con tres caras: una **pantalla pública** para el ciudadano (mapa de la red, horarios, recarga de tarjeta y buzón de sugerencias), un **portal del pasajero** y un **panel del personal** con permisos por rol (administrador, personal municipal, operador de estación y guardia).

Indicador	Resultado
Requerimientos funcionales	8 de 12 completos; RF-5, RF-8 y RF-9 con el software listo y el hardware o proveedor externo por integrar; RF-12 en fase 2, con la idempotencia de cobros ya construida
Requerimientos no funcionales	RNF-2, RNF-4 y RNF-6 completos; RNF-1, RNF-3, RNF-5 y RNF-7 parciales (ver sección 7)
Pruebas automatizadas	103 de 103 aprobadas (Jest + Supertest)
Casos del plan de pruebas ejecutados	55 de 60; 54 aprobados y 1 fallido (CP-57, vulnerabilidad abierta)
Cobertura de código del backend	93.7 % de líneas; 98 % a 100 % en los servicios con las reglas de negocio
Latencia de posiciones en tiempo real (p95)	7 ms en entorno local (meta: 2 s o menos)
Tiempo de respuesta de la API (p95)	284 ms en entorno local (meta: 500 ms o menos)
Respaldo y restauración	Comprobados con pg_dump y pg_restore: datos idénticos en menos de 1 segundo
Publicación	Render (aplicación) y Neon (PostgreSQL), plan gratuito
Inversión estimada de la solución real (fase piloto)	Q 1,314,000.00 (aprox. USD 168,500), ver sección 3

Tecnología. React y Tailwind CSS en el navegador; API REST en Node.js con Express 5 y TypeScript; PostgreSQL con Sequelize; Socket.IO para el tiempo real; Leaflet con OpenStreetMap para los mapas; JWT, bcrypt, Zod y AES-256-GCM para la seguridad.

Conclusión. El sistema cumple las reglas de negocio centrales del enunciado y está verificado con pruebas trazables a cada requerimiento. Quedan como trabajo pendiente la operación sin conexión, las pruebas de extremo a extremo y de carga con herramientas dedicadas, la documentación OpenAPI, la integración continua y la actualización de una dependencia con vulnerabilidad alta.

1

Análisis de requerimientos

Actores del sistema, los 12 requerimientos funcionales y los 7 no funcionales que guían el diseño, la construcción y las pruebas.

Documento entregado el 21-08-2026.

Actores del sistema:

- Pasajero / Usuario del sistema
- Piloto
- Gerente de estación
- Guardia de seguridad asignado a un acceso
- Administrador central del sistema
- Personal de la muni de guate
- Sistema de pago

Requerimientos funcionales

1. Registrar cada línea con el orden de sus estaciones (una estación puede pertenecer a varias lineas), la secuencia de visitas de cada unidad durante el recorrido, la distancia entre estaciones y la distancia total de la línea, y la municipalidad a al que pertenecen la linea y cada estación, siguiendo la práctica de sistemas como los de colombia que es Transmilenio o la RIT de Curitiba
2. Registrar uno o varios accesos por estación con al menos un guardia de seguridad asignado, y designar un operador responsable por estación con acceso a una cosola conectada mediante red al servidor central del sistema
3. Asignar cada bus a una única línea, validando que el número de buses por línea se mantenga entre el mínimo (igual al numero de estaciones) y el maximo (el doble)
4. Registrar la capacidad máxima de pasajeros de cada bus y los parqueos disponibles, asociándose a las estaciones y asignándoles a cada unidad, permitiendo su reasignación sin que quede sin parqueo.
5. Incorporar seguimiento por geolocalización (GPS/AVL) de cada unidad, siguiendo el modelo de monitoreo de flota en su tiempo real de sistemas como el transmilenio de bogotá
6. Registrar el historial educativo y los datos de residencia y comunicación de cada piloto y asociarlo con el bus y la linea en la que presta servicio
7. Calcular la ocupación de cada bus para que nos diga cuando ya esta sobre su capacidad para que se despache una unidad adicional y extendiendo 5 min la espera cuando la ocupación es de 25 por ciento
8. Poner camaras fifjas en estaciones /accesos y a bordo de las unidades, registrar incidentes de seguridad y contar con un botón de panico con notificación inmediata al centro de control
9. soportar el pago mediante tarjeta de acceso
10. Generar reportes de estaciones y línea de buses asignados a cada una exportables para su análisis
11. Generar tableros de control
12. Contemplar almacenamiento y sincronización local ante perdida de conectividad, replicando la información al establecer la conexion

Requerimientos no funcionales

1. El sistema debe estar disponible las 24 horas del dia
2. Debe procesar aletar de capacidad y actualizaciones de posicion de los buses en tiempo real

3. El sistema debe cifrar la informacion sensible de pilotos y las transacciones de recaudo y restringir el acceso mediante roles
4. Debe soportar el crecimiento en el numero de lineas
5. Debe permitir una integracion futura con otros sistemas de transporte
6. Debe registrar un historial de cambios sobre las asignaciones de buses
7. Debe contar con mecanismos de respaldo y recuperacion ante fallos

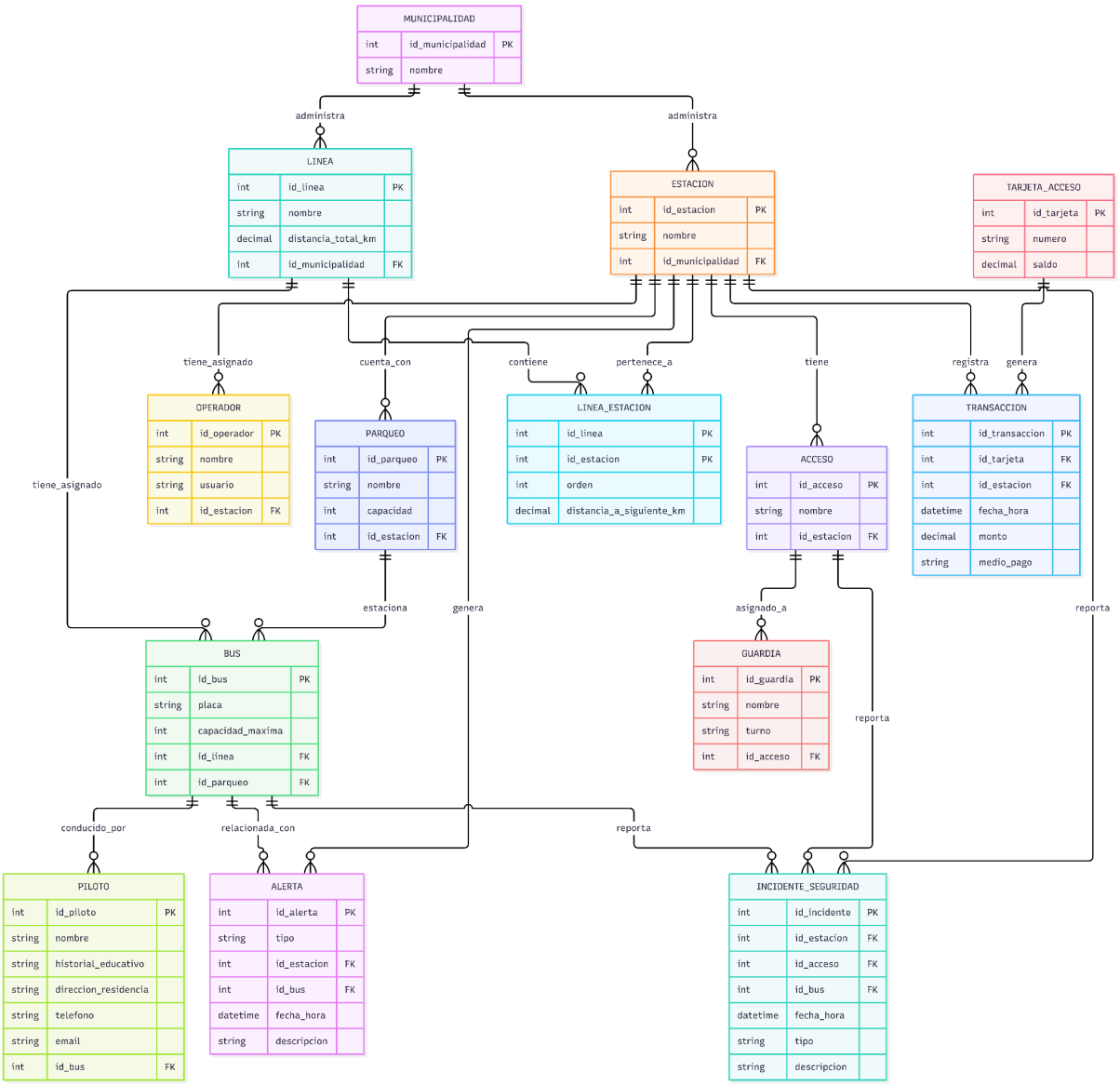
2

Diseño del sistema

Modelo entidad-relación, diagramas UML (casos de uso, componentes, secuencia, actividades, clases y objetos) y script de creación de la base de datos.

Documento entregado el 21-08-2026. El script original está escrito para MySQL; su versión en PostgreSQL está en el anexo A.

ER:



Casos de uso:

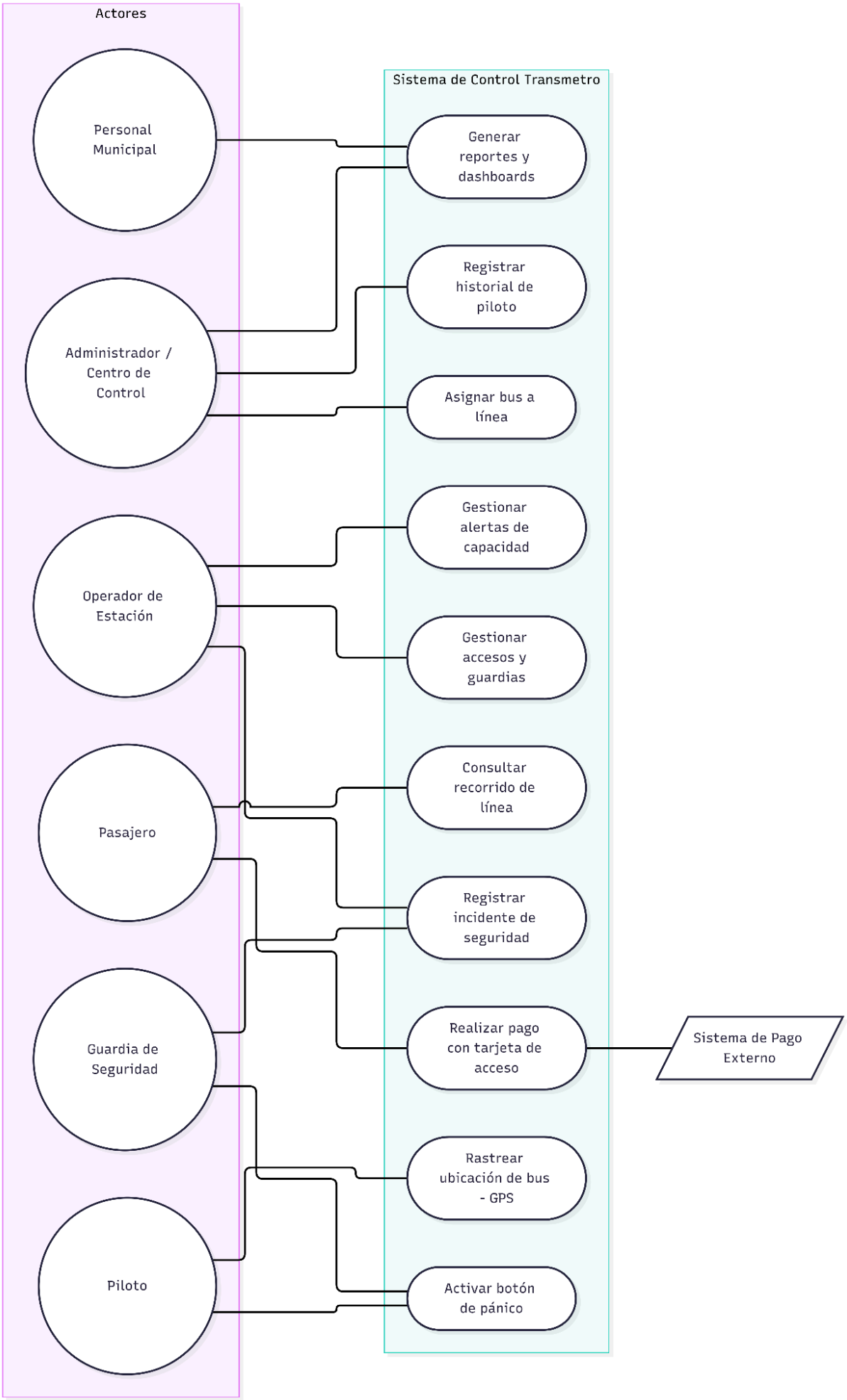
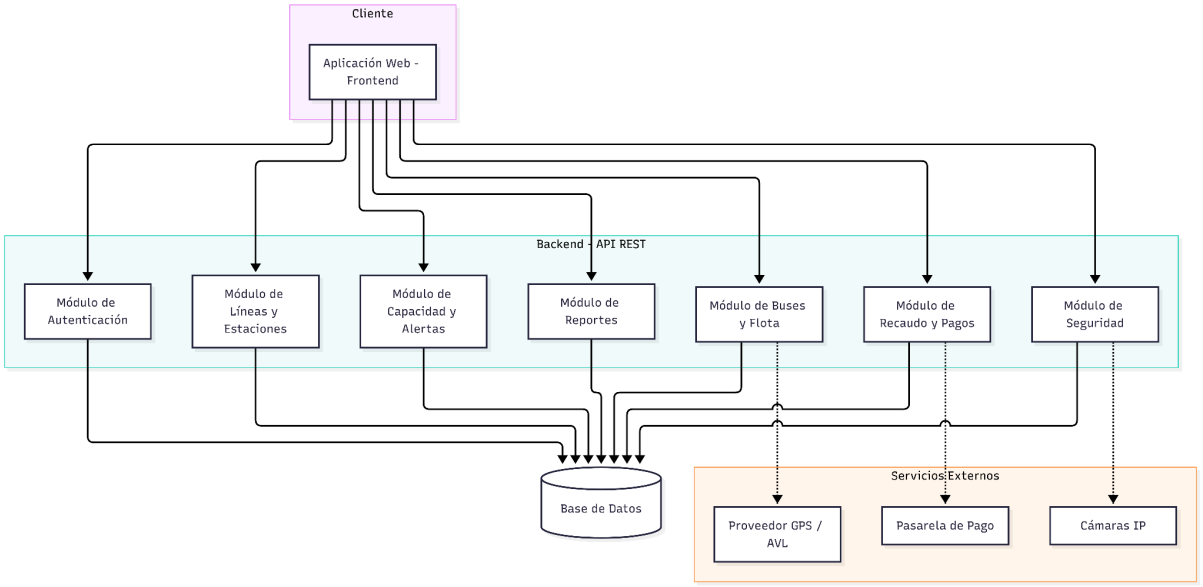
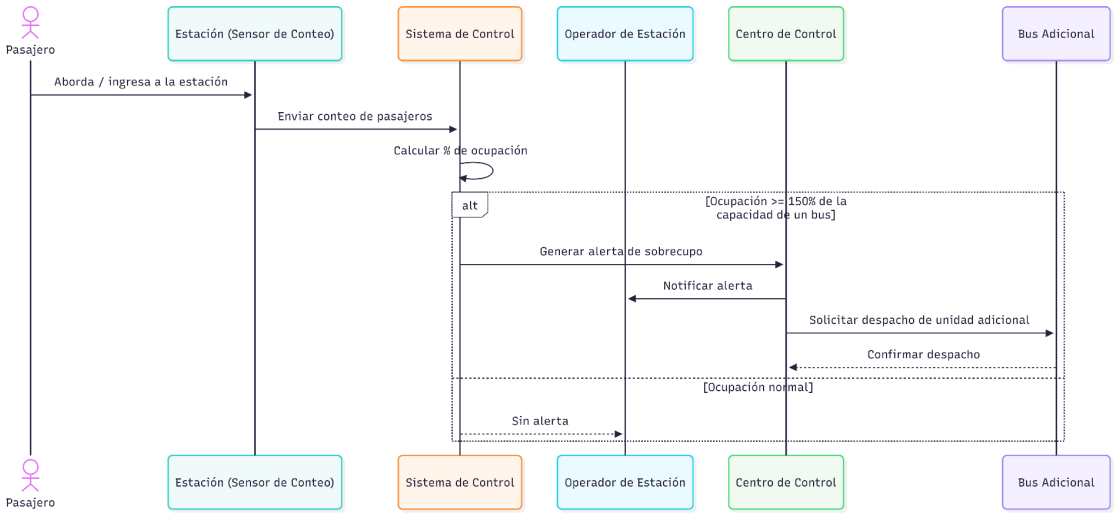

Actores
Personal Municipal
Administrador / Centro de Control
Operador de Estación
Pasajero
Guardia de Seguridad
Piloto
Sistema de Control Transmetro
Generar reportes y dashboards
Registrar historial de piloto
Asignar bus a línea
Gestionar alertas de capacidad
Gestionar accesos y guardias
Consultar recorrido de línea
Registrar incidente de seguridad
Realizar pago con tarjeta de acceso
Rastrear ubicación de bus - GPS
Activar botón de pánico
Sistema de Pago Externo

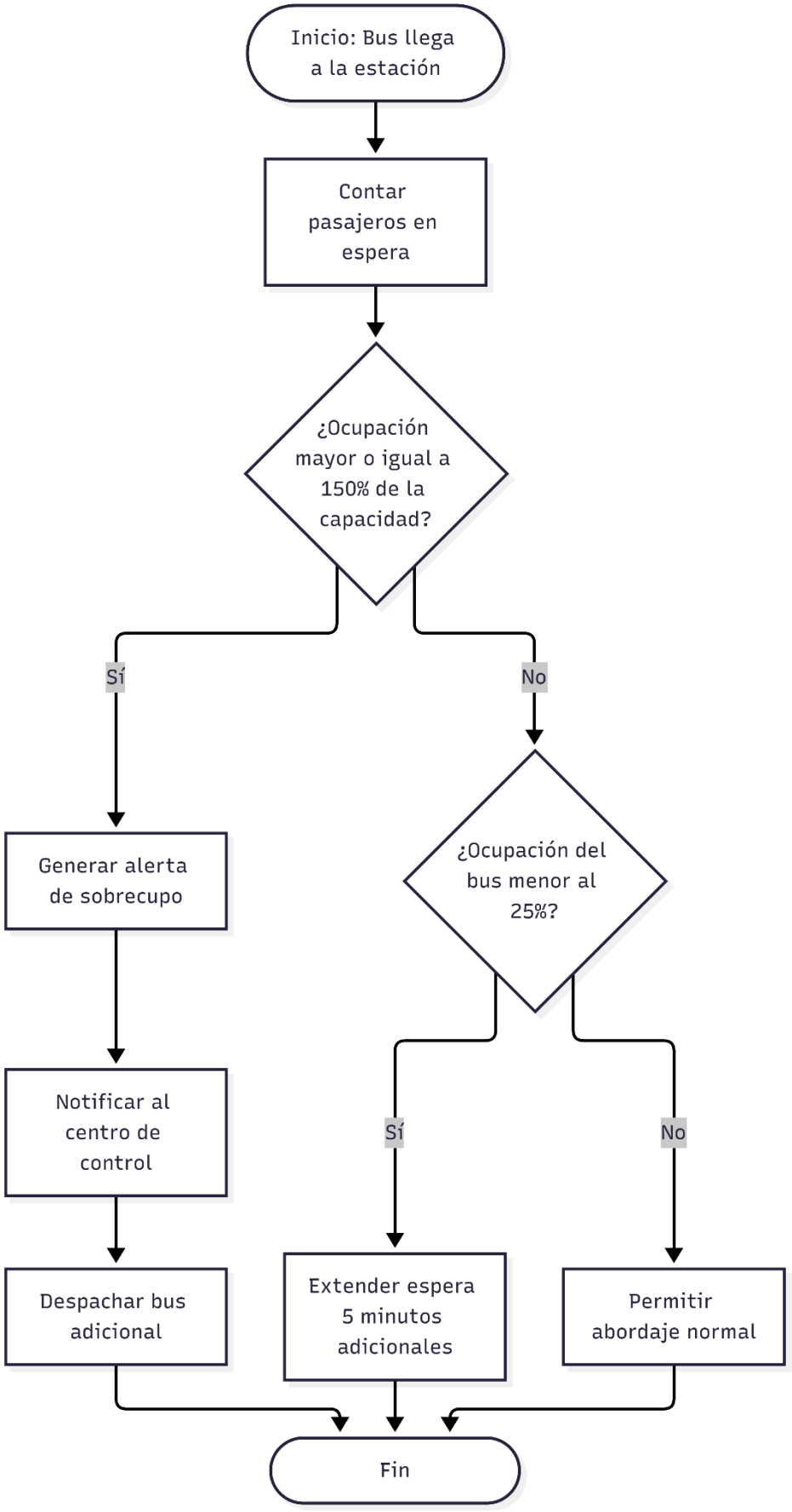
Diagrama de componentes



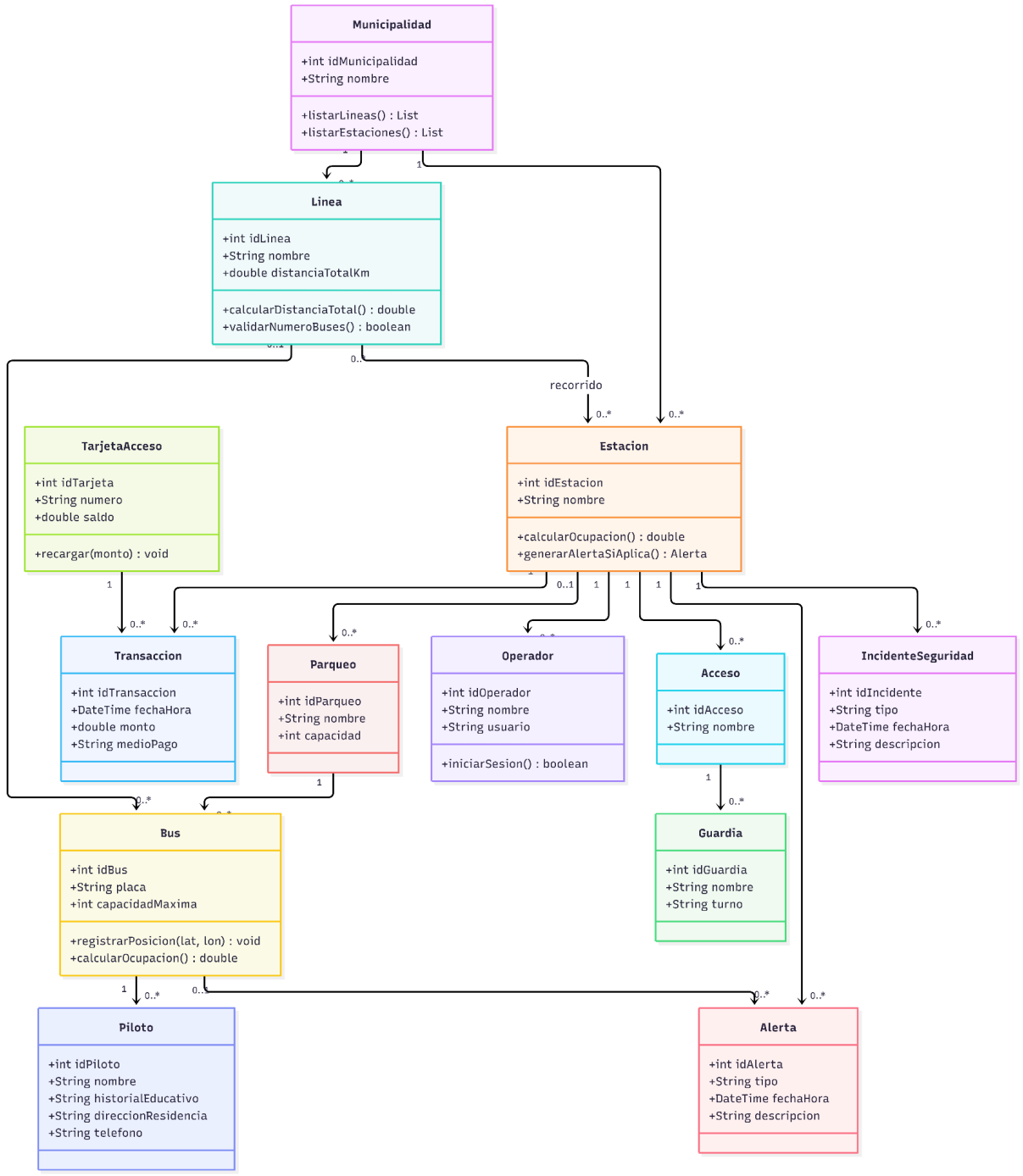
Secuencia



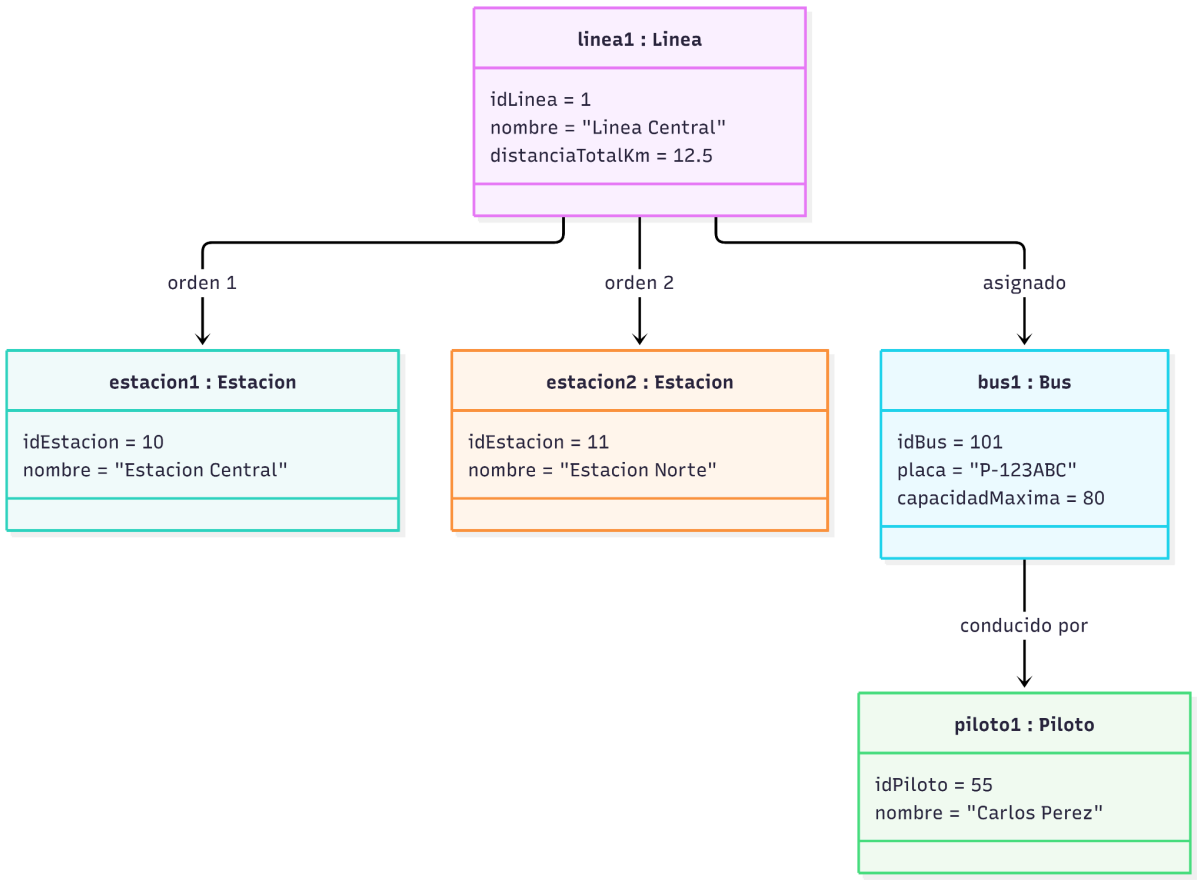
actividades:


Inicio: Bus llega a la estación
Contar pasajeros en espera
¿Ocupación mayor o igual a 150% de la capacidad?
Sí
No
Generar alerta de sobrecupo
Notificar al centro de control
Despachar bus adicional
¿Ocupación del bus menor al 25%?
Sí
No
Extender espera 5 minutos adicionales
Permitir abordaje normal
Fin

clases:



Objetos



Scripts:

```
CREATE DATABASE IF NOT EXISTS transmetro_db;

USE transmetro_db;

CREATE TABLE municipalidad (
    id_municipalidad INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE linea (
    id_linea INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    distancia_total_km DECIMAL(6,2),
    id_municipalidad INT NOT NULL,
    FOREIGN KEY (id_municipalidad) REFERENCES municipalidad(id_municipalidad)
);

CREATE TABLE estacion (
    id_estacion INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    id_municipalidad INT NOT NULL,
    FOREIGN KEY (id_municipalidad) REFERENCES municipalidad(id_municipalidad)
);

CREATE TABLE linea_estacion (
    id_linea INT NOT NULL,
    id_estacion INT NOT NULL,
    orden INT NOT NULL,
    distancia_a_siguiente_km DECIMAL(6,2),
    PRIMARY KEY (id_linea, id_estacion),
    FOREIGN KEY (id_linea) REFERENCES linea(id_linea),
    FOREIGN KEY (id_estacion) REFERENCES estacion(id_estacion)
```

```
);

CREATE TABLE acceso (
    id_acceso INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    id_estacion INT NOT NULL,
    FOREIGN KEY (id_estacion) REFERENCES estacion(id_estacion)
);

CREATE TABLE guardia (
    id_guardia INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    turno VARCHAR(20),
    id_acceso INT NOT NULL,
    FOREIGN KEY (id_acceso) REFERENCES acceso(id_acceso)
);

CREATE TABLE operador (
    id_operador INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    usuario VARCHAR(50) NOT NULL UNIQUE,
    id_estacion INT NOT NULL,
    FOREIGN KEY (id_estacion) REFERENCES estacion(id_estacion)
);

CREATE TABLE parqueo (
    id_parqueo INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    capacidad INT,
    id_estacion INT,
    FOREIGN KEY (id_estacion) REFERENCES estacion(id_estacion)
);

CREATE TABLE bus (
    id_bus INT AUTO_INCREMENT PRIMARY KEY,
    placa VARCHAR(15) NOT NULL UNIQUE,
    capacidad_maxima INT NOT NULL,
    id_linea INT,
    id_parqueo INT,
    FOREIGN KEY (id_linea) REFERENCES linea(id_linea),
    FOREIGN KEY (id_parqueo) REFERENCES parqueo(id_parqueo)
);

CREATE TABLE piloto (
    id_piloto INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    historial_educativo VARCHAR(255),
    direccion_residencia VARCHAR(255),
    telefono VARCHAR(20),
    email VARCHAR(100),
    id_bus INT,
    FOREIGN KEY (id_bus) REFERENCES bus(id_bus)
);

CREATE TABLE tarjeta_acceso (
    id_tarjeta INT AUTO_INCREMENT PRIMARY KEY,
    numero VARCHAR(30) NOT NULL UNIQUE,
    saldo DECIMAL(8,2) DEFAULT 0
);

CREATE TABLE transaccion (
    id_transaccion INT AUTO_INCREMENT PRIMARY KEY,
    id_tarjeta INT NOT NULL,
    id_estacion INT NOT NULL,
    fecha_hora DATETIME NOT NULL,
    monto DECIMAL(8,2) NOT NULL,
    medio_pago VARCHAR(30) NOT NULL,
    FOREIGN KEY (id_tarjeta) REFERENCES tarjeta_acceso(id_tarjeta),
    FOREIGN KEY (id_estacion) REFERENCES estacion(id_estacion)
);
```

```
CREATE TABLE alerta (
    id_alerta INT AUTO_INCREMENT PRIMARY KEY,
    tipo VARCHAR(30) NOT NULL,
    id_estacion INT,
    id_bus INT,
    fecha_hora DATETIME NOT NULL,
    descripcion VARCHAR(255),
    FOREIGN KEY (id_estacion) REFERENCES estacion(id_estacion),
    FOREIGN KEY (id_bus) REFERENCES bus(id_bus)
);

CREATE TABLE incidente_seguridad (
    id_incidente INT AUTO_INCREMENT PRIMARY KEY,
    id_estacion INT,
    id_acceso INT,
    id_bus INT,
    fecha_hora DATETIME NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    descripcion VARCHAR(255),
    FOREIGN KEY (id_estacion) REFERENCES estacion(id_estacion),
    FOREIGN KEY (id_acceso) REFERENCES acceso(id_acceso),
    FOREIGN KEY (id_bus) REFERENCES bus(id_bus)
);
```

3

Estudios de factibilidad y propuesta monetaria

Factibilidad técnica, operativa, económica, legal y ambiental; desglose de la inversión, calendario de pagos y modalidad de contratación.

Documento entregado el 21-08-2026 (secciones 7 y 8 de la propuesta).

7. Estudios de Factibilidad

El presente estudio evalúa la viabilidad del proyecto desde cinco dimensiones — técnica, operativa, económica, legal y ambiental — con el fin de sustentar la decisión de las municipalidades involucradas de avanzar con la implementación del Sistema de Control de Líneas, Estaciones y Buses del Transmetro.

7.1 Factibilidad Técnica

Evalúa si existe la capacidad tecnológica, la infraestructura y el talento humano necesarios para desarrollar e implementar la solución propuesta.

- Infraestructura de hardware: una PC de control robusta por estación conectada a la red del sistema, cámaras IP fijas en accesos y estaciones, cámaras a bordo de cada unidad, dispositivos de geolocalización (GPS/AVL) en los buses, y validadores de tarjeta de acceso en torniquetes, puertas y unidades.
- Arquitectura de software: aplicación web con arquitectura cliente-servidor desplegada en un proveedor de nube (AWS, Azure o Google Cloud), que permite escalar el número de líneas, estaciones y municipalidades sin reinversión significativa en infraestructura física; base de datos relacional que garantiza la integridad de las reglas de negocio entre líneas, estaciones y buses; APIs documentadas para la integración de dispositivos GPS/AVL, cámaras y validadores de pago.
- Recurso humano técnico: perfiles de desarrollo full-stack, administración de base de datos, aseguramiento de la calidad (QA) y administración de infraestructura, todos disponibles razonablemente en el mercado laboral guatemalteco, particularmente en la ciudad capital.
- Interoperabilidad: las tecnologías propuestas (GPS/AVL, cámaras IP, validadores NFC) son estándares de la industria con múltiples proveedores locales e internacionales, lo que reduce el riesgo de dependencia de un único fabricante.
- Riesgos técnicos identificados: interrupciones de conectividad entre estaciones remotas y el servidor central (mitigado mediante el requerimiento de almacenamiento y sincronización local, RF-12) y la integración de hardware heterogéneo (mitigado mediante el uso de estándares abiertos y APIs documentadas).

Conclusión: el proyecto es técnicamente factible, ya que se apoya en tecnologías maduras y ampliamente probadas en sistemas BRT similares (geolocalización de flota, cámaras IP, conteo automático de pasajeros y recaudo con tarjeta inteligente), sin depender de desarrollos experimentales.

7.2 Factibilidad Operativa

Evalúa si las municipalidades y su personal están en condiciones de operar el sistema una vez implementado.

- Capacidad organizacional: las municipalidades ya cuentan con personal operativo (guardias, pilotos, personal administrativo) que puede asumir los nuevos roles definidos (operador de estación, administrador del sistema) mediante la capacitación adecuada.
- Aceptación de los usuarios: se espera una curva de adopción gradual; los pilotos y guardias deberán familiarizarse con herramientas digitales, por lo que se requiere un plan de gestión del cambio (comunicación, sensibilización e incentivos).

- Capacitación: un plan diferenciado por rol (operadores de estación, pilotos, guardias de seguridad y administradores del centro de control), con manuales de usuario y sesiones prácticas previas a la puesta en marcha.
- Soporte post-implementación: se recomienda una mesa de ayuda (help desk) durante los primeros meses de operación para resolver incidentes y dudas operativas.
- Gobernanza interinstitucional: dado que el sistema involucra a varias municipalidades, se requiere un acuerdo de gobernanza compartida — posiblemente mediante una mancomunidad — que defina responsabilidades sobre la operación diaria y el mantenimiento.
- Riesgos operativos: resistencia al cambio y rotación de personal (pilotos); mitigados mediante un programa de capacitación permanente y documentación accesible.

Conclusión: el proyecto es operativamente factible siempre que se acompañe de un plan formal de capacitación, gestión del cambio y gobernanza compartida entre las municipalidades participantes.

7.3 Factibilidad Económica

Evalúa si los beneficios esperados del proyecto justifican la inversión requerida, detallada en la Propuesta Monetaria de la Solución (sección 8).

- Beneficios cuantificables: incremento del recaudo formal al reducir las fugas asociadas al manejo manual de efectivo, reducción de costos operativos por una mejor asignación y aprovechamiento de la flota, y disminución de gastos asociados a incidentes de seguridad no reportados oportunamente.
- Beneficios cualitativos: mejora en la percepción de seguridad de los usuarios (factor que, según la experiencia de sistemas BRT similares, incrementa la demanda y la aceptación del transporte formal), mejor toma de decisiones gracias a los reportes y tableros de control, y fortalecimiento institucional de las municipalidades.
- Fuentes de financiamiento: presupuesto municipal ordinario, fondos de una mancomunidad intermunicipal, préstamos de organismos multilaterales (BID, Banco Mundial) que históricamente han financiado proyectos de movilidad urbana en la región, y esquemas de cooperación público-privada.
- Marco de análisis costo-beneficio: se recomienda que las municipalidades validen esta propuesta con cifras reales de recaudo y costos operativos actuales, de forma que pueda calcularse un periodo de retorno de la inversión específico. De manera referencial, si la formalización del recaudo y la optimización de flota generan ahorros o ingresos adicionales de forma sostenida, el retorno de la inversión podría alcanzarse en un plazo de 3 a 5 años, horizonte razonable para proyectos de infraestructura tecnológica municipal.

Conclusión: el proyecto es económicamente factible, considerando que varios de sus beneficios ya están documentados en sistemas BRT similares a nivel internacional, y que existen múltiples fuentes de financiamiento disponibles para proyectos de esta naturaleza.

7.4 Factibilidad Legal

Evalúa el marco normativo guatemalteco aplicable a la contratación, implementación y operación del sistema.

- Contratación pública: la adquisición de bienes y servicios por parte de las municipalidades se rige por la Ley de Contrataciones del Estado (Decreto 57-92 del Congreso de la República) y debe registrarse en el sistema Guatecompras. Según el Artículo 38 de dicha ley, las contrataciones municipales entre Q90,000.00 y Q900,000.00 pueden tramitarse por el sistema de cotización; montos superiores requieren licitación pública, por lo que, dado el monto estimado de esta propuesta, se recomienda evaluar una licitación pública o una contratación por fases/municipio que se ajuste a los umbrales vigentes.
- Modernización normativa: al momento de esta propuesta, el Ministerio de Finanzas Públicas impulsa la iniciativa de ley 6688, que busca sustituir la Ley de Contrataciones vigente desde 1992, digitalizar por completo las compras públicas y convertir Guatecompras en una plataforma transaccional con trazabilidad completa; se recomienda dar seguimiento a este proceso legislativo durante la ejecución del proyecto.
- Marco municipal: el Código Municipal (Decreto 12-2002) faculta a las municipalidades a contratar servicios y a asociarse mediante mancomunidades, figura recomendable para administrar de forma conjunta un sistema que opera entre la capital y varios municipios aledaños.
- Normativa de tránsito y transporte: la operación de las líneas, estaciones y buses debe respetar la Ley de Tránsito y los reglamentos municipales de transporte vigentes en cada jurisdicción.
- Protección de datos personales: Guatemala aún no cuenta con una ley integral de protección de datos personales en vigor; el resguardo de datos personales se sustenta actualmente en el derecho constitucional de acceso a la información personal (hábeas data) y en las disposiciones sobre datos personales de la Ley de Acceso a la Información Pública (Decreto 57-2008). Existe una iniciativa de ley específica de protección de datos personales en discusión en el Congreso de la República, aún no aprobada. Por ello, se recomienda que el sistema adopte desde su diseño buenas prácticas internacionales de protección de datos (principio de minimización, cifrado, control de acceso por roles) de forma preventiva, tanto para los datos de los pilotos como para las transacciones de recaudo.
- Propiedad intelectual: el contrato de consultoría debe definir con claridad la titularidad del código fuente, la documentación técnica y las licencias de los componentes de terceros utilizados.

Conclusión: el proyecto es legalmente factible, sujeto al cumplimiento de la normativa de contratación pública vigente (evaluando la modalidad adecuada según el monto final) y a la adopción proactiva de buenas prácticas de protección de datos mientras el país no cuente con una ley integral en la materia.

7.5 Factibilidad Ambiental

Evalúa el impacto ambiental del proyecto, tanto directo (infraestructura tecnológica) como indirecto (efecto sobre el comportamiento de movilidad de los usuarios).

- Evidencia internacional de reducción de emisiones: el sistema BRT de Guangzhou (China) evita la emisión de 84,000 toneladas de CO2 al año desde su implementación en 2010, cifra que se estima podría casi triplicarse con la electrificación de la flota.
- Eficiencia operativa y ambiental conjunta: los sistemas BRT permiten una circulación más rápida que el transporte convencional (por ejemplo, TransMilenio en Bogotá circula en promedio 10 km/h más rápido que los buses urbanos regulares), lo que reduce el consumo de combustible por pasajero transportado.

- Cambio modal: estudios en la Zona Metropolitana del Valle de México muestran que los usuarios que sustituyen el automóvil particular por un servicio BRT (Metrobús/Mexibús) reducen hasta un 95% sus emisiones de gases de efecto invernadero, mientras que quienes migran de microbuses o transporte informal a un sistema BRT logran reducciones de hasta un 56% en emisiones de CO2.
- Beneficios en salud pública: la mejora en la calidad del aire asociada a corredores BRT ha evitado miles de días de trabajo perdidos por enfermedad en ciudades como la Ciudad de México, un beneficio indirecto pero relevante para la factibilidad ambiental y social del proyecto.
- Gestión del ciclo de vida del hardware: los equipos instalados (PC, cámaras, dispositivos GPS/AVL) deberán seguir un plan de disposición final o reciclaje certificado al concluir su vida útil, en línea con buenas prácticas de gestión de desechos electrónicos.
- Infraestructura en la nube: el uso de servicios de nube para el backend y la base de datos central reduce el consumo energético frente a mantener servidores propios en cada municipalidad, al compartir infraestructura con mayor eficiencia energética.
- Alineación con los Objetivos de Desarrollo Sostenible (ODS): el proyecto contribuye directamente al ODS 11 (Ciudades y Comunidades Sostenibles) y al ODS 13 (Acción por el Clima), al fortalecer un sistema de transporte público formal, seguro y eficiente frente a alternativas informales y al uso del vehículo particular.

Conclusión: el proyecto es ambientalmente factible; su impacto directo (consumo energético de la infraestructura tecnológica) es reducido y manejable, mientras que su impacto indirecto —al incentivar el cambio modal hacia un transporte público más seguro y confiable— es claramente positivo, conforme a la evidencia documentada en sistemas BRT comparables.

7.6 Conclusión General de los Estudios de Factibilidad

Las cinco dimensiones analizadas — técnica, operativa, económica, legal y ambiental — resultan favorables para la implementación del Sistema de Control de Líneas, Estaciones y Buses del Transmetro. Se recomienda a las municipalidades avanzar con el proyecto, sujeto a: (1) definir la modalidad de contratación adecuada según el monto final de inversión, (2) formalizar un esquema de gobernanza compartida entre las municipalidades participantes, y (3) incorporar desde el diseño buenas prácticas de protección de datos y gestión ambiental del hardware.

8. Propuesta Monetaria de la Solución

La presente propuesta monetaria se elabora sobre un alcance inicial de referencia (fase piloto) de 3 líneas, 25 estaciones y 60 buses, alineado con la regla de negocio que exige entre 25 y 50 buses por línea para ese número de estaciones. Este alcance es ajustable según el número real de líneas, estaciones y buses que las municipalidades definan para la primera fase de implementación, y los montos deberán revalidarse en ese caso.

8.1 Desglose de la inversión

Rubro	Descripción	Costo estimado (GTQ)
Análisis y diseño	Levantamiento de requerimientos, Modelo Entidad-Relación y diagramas UML (casos de uso, clases, secuencia, actividades, componentes).	Q 45,000.00
Desarrollo de software	Backend, frontend, base de datos y APIs de integración con GPS/AVL, cámaras y validadores de pago.	Q 340,000.00
Infraestructura y hosting	Servicios en la nube (aplicación y base de datos central) durante el primer año de operación.	Q 65,000.00
Hardware por estación	PC de control, cámara IP y validador de tarjeta de acceso, para 25 estaciones (Q15,000.00 c/u).	Q 375,000.00
Dispositivos GPS/AVL	Equipo de geolocalización para 60 buses de la flota inicial (Q2,800.00 c/u).	Q 168,000.00
Integración de recaudo	Plataforma de pago con tarjeta inteligente y pasarela de conciliación de transacciones.	Q 85,000.00
Estándares de seguridad	Cifrado de datos, control de acceso por roles y registro de auditoría.	Q 40,000.00
Plan de pruebas de calidad	Pruebas funcionales, de integración, de carga y de aceptación.	Q 38,000.00
Capacitación	Talleres para operadores de estación, pilotos, guardias de seguridad y administradores del centro de control.	Q 42,000.00
Documentación y garantía	Manuales técnicos y de usuario, entrega formal, y 3 meses de garantía posteriores a la implementación.	Q 20,000.00
Soporte y mantenimiento	Mesa de ayuda y mantenimiento correctivo/evolutivo durante los 12 meses posteriores a la entrega.	Q 96,000.00

INVERSIÓN TOTAL ESTIMADA	**Q 1,314,000.00**

La inversión total estimada equivale aproximadamente a USD 168,500.00, utilizando un tipo de cambio referencial de Q7.80 por dólar. Este monto no incluye la renovación del soporte y mantenimiento a partir del segundo año de operación, la cual se recomienda cotizar de forma anual conforme al desempeño y crecimiento del sistema.

8.2 Calendario de pagos propuesto

Se propone un esquema de pagos por hitos, alineado con la planificación descrita en la sección 6, que vincula cada desembolso a un entregable verificable:

Hito de pago	Momento	% del total	Monto (GTQ)
Anticipo a la firma del contrato	Semana 1	20%	Q 262,800.00
Entrega del Modelo Entidad-Relación y diagramas UML	Semana 4	20%	Q 262,800.00

Hito de pago	Momento	% del total	Monto (GTQ)
Aplicación web funcional y hardware instalado en estaciones piloto	Semana 8	35%	Q 459,900.00
Aceptación final (pruebas de calidad y capacitación completadas)	Semana 10	25%	Q 328,500.00

8.3 Consideraciones sobre la modalidad de contratación

Dado que la inversión total estimada supera el umbral de Q900,000.00 establecido en el Artículo 38 de la Ley de Contrataciones del Estado para el sistema de cotización municipal, se recomienda a las municipalidades evaluar dos alternativas: (1) tramitar el proyecto como licitación pública, dada su cuantía, o (2) dividir la contratación por fases o por municipio (por ejemplo, iniciando con una sola línea piloto), de manera que cada contratación individual se ajuste a los montos permitidos por el sistema de cotización, sujeto a la evaluación legal correspondiente.

-

4

Stack tecnológico y métricas de calidad

Tecnologías elegidas y su justificación, trazabilidad con los requerimientos, alternativas consideradas y métricas de producto, pruebas, rendimiento y proceso alineadas con ISO/IEC 25010.

Documento entregado el 21-10-2026. La sección 5 compara este stack con lo implementado.

1. Introducción

Este documento define el stack tecnológico del Sistema de Control de Líneas, Estaciones y Buses del Transmetro

2. Resumen del stack

Categoría	Tecnología	Función en el proyecto
Base de datos	PostgreSQL + Sequelize (ORM y migraciones)	Persistencia relacional, integridad de reglas de negocio, auditoría y respaldos.
Metodología	Scrum adaptado a 10 semanas (5 sprints)	Entregas incrementales alineadas con los hitos de pago (semanas 4, 8 y 10).
Lenguaje	TypeScript (Node.js) y SQL	Un solo lenguaje tipado en backend y frontend.
Framework backend	Node.js + Express 5	API REST modular y manejo asíncrono de eventos.
Framework frontend	React + Tailwind CSS	Consolas por rol, tableros de control y vistas de mapa.
IDE	Visual Studio Code	Desarrollo, depuración e integración con Git y Docker.
Tiempo real	Socket.IO (WebSockets)	Posición GPS/AVL y alertas de capacidad en vivo.
Seguridad	JWT, bcrypt, Zod, RBAC, TLS, cifrado de columnas	Autenticación, control de acceso por roles y protección de datos.
Mapas y reportes	Leaflet + OpenStreetMap, Recharts, exportación XLSX/CSV	Rastreo de flota, tableros y reportes exportables.
Pruebas y métricas	Jest, Supertest, Playwright, k6, SonarQube	Pruebas funcionales, de integración, de carga y métricas de calidad.
DevOps y nube	Docker Compose, Git/GitHub, GitHub Actions, AWS EC2 + Nginx	Entorno reproducible, integración continua y despliegue.
Documentación de API	Swagger / OpenAPI	Contrato documentado para GPS/AVL, cámaras y validadores.

3. Justificación por categoría

3.1 Base de datos: PostgreSQL con Sequelize

El modelo del Transmetro es profundamente relacional: municipalidad, línea, estación, línea_estación, acceso, guardia, operador, parqueo, bus, piloto, tarjeta_acceso, transacción, alerta e incidente_seguridad. Un motor relacional es la elección natural, y PostgreSQL aporta lo que las reglas del proyecto exigen:

- Reglas de negocio en la capa de datos. Restricciones, claves foráneas y triggers permiten hacer cumplir que un bus pertenezca a una sola línea, que el número de buses por línea esté entre el número de estaciones y el doble (RF-3), y que un bus nunca quede sin parqueo al reasignarlo (RF-4).
- Transacciones ACID. Necesarias para el recaudo con tarjeta (RF-9): descontar saldo y registrar la transacción deben ocurrir juntos o no ocurrir.
- Historial de cambios. El RNF-6 exige trazabilidad de las asignaciones de buses; se resuelve con tablas de auditoría alimentadas por triggers o por la capa de servicios.
- Cifrado y respaldo. La extensión pgcrypto permite cifrar datos sensibles de pilotos (RNF-3). El archivado de WAL y pg_dump programado cubren el respaldo y la recuperación ante fallos (RNF-7).
- Crecimiento y datos geográficos. Soporta el aumento de líneas (RNF-4) y puede ampliarse con PostGIS para consultas espaciales de posiciones GPS si el volumen lo justifica.
- Sequelize. Modelos, relaciones, migraciones versionadas y transacciones gestionadas, igual que en Hunahpú. Las migraciones permiten evolucionar el esquema sin perder control de versiones.

Nota de adaptación: el script SQL entregado (transmetro_db) está escrito para MySQL (AUTO_INCREMENT, DATETIME, USE). Para PostgreSQL se sustituye por columnas IDENTITY o SERIAL y TIMESTAMP, y se eliminan CREATE DATABASE IF NOT EXISTS y USE. El modelo lógico y las relaciones del ER no cambian.

3.2 Metodología: Scrum adaptado

El proyecto se ejecuta en 10 semanas con pagos atados a entregables verificables (semanas 4, 8 y 10). Una metodología puramente en cascada llegaría tarde a mostrar software funcional; una puramente ágil sin hitos no encaja con el contrato. Por eso se propone Scrum adaptado:

- Cinco sprints de dos semanas. El sprint 2 cierra con el hito de la semana 4 (Modelo ER y diagramas UML), el sprint 4 con el hito de la semana 8 (aplicación funcional) y el sprint 5 con la aceptación final en la semana 10.
- Product backlog = requerimientos. Los 12 RF y los 7 RNF se convierten en historias de usuario priorizadas junto con las municipalidades.
- Calidad integrada. La definición de terminado incluye pruebas, revisión de código y documentación, de modo que el aseguramiento de la calidad (objetivo del curso) ocurre durante todo el ciclo y no solo al final.

- Retroalimentación frecuente. Las revisiones de sprint con operadores, guardias y personal municipal reducen el riesgo operativo de resistencia al cambio identificado en la factibilidad.

3.3 Lenguaje: TypeScript

- Tipado estático. Reduce errores en un dominio con muchas entidades relacionadas y reglas numéricas (capacidades, porcentajes de ocupación, saldos).
- Un solo lenguaje de extremo a extremo. Los tipos y contratos (por ejemplo, el formato de una alerta o de una posición GPS) se comparten entre backend y frontend dentro del monorepositorio.
- Mantenibilidad medible. Facilita las métricas de software (complejidad, duplicación, deuda técnica) y el mantenimiento futuro que exige la propuesta.
- Disponibilidad de talento. Los perfiles full-stack en JavaScript/TypeScript son abundantes en el mercado guatemalteco, como se indicó en la factibilidad técnica.
- SQL. Se emplea para el script de creación de la base de datos, vistas de reportes y triggers de validación.

3.4 Framework backend: Node.js y Express 5

- Modularidad. Cada módulo del diagrama de componentes (Autenticación, Líneas y Estaciones, Capacidad y Alertas, Reportes, Buses y Flota, Recaudo y Pagos, Seguridad) se implementa como un conjunto de rutas, controladores y servicios independientes.
- Modelo asíncrono. El sistema es intensivo en entrada/salida: cientos de unidades enviando posiciones, validadores reportando transacciones y consolas consultando datos. Node.js maneja bien esa concurrencia (RNF-2).
- Patrón por capas. Se mantiene el de Hunahpú: Routes, Validators, Middlewares, Controllers, Services y Models. Toda la lógica de negocio (cálculo de ocupación, validación de rangos de buses) vive en Services, lo que la hace probable de forma aislada.
- API documentada. Express junto con Swagger/OpenAPI produce el contrato de integración para GPS/AVL, cámaras IP y validadores, y habilita la integración futura con otros sistemas de transporte (RNF-5).

3.5 Framework frontend: React y Tailwind CSS

- Consolas por rol. Operador de estación, administrador del centro de control, personal municipal y guardias necesitan vistas distintas; React con rutas protegidas lo resuelve con componentes reutilizables.
- Tableros y reportes. Recharts para los tableros de control (RF-11) y exportación a XLSX/CSV para los reportes de estaciones y líneas (RF-10).
- Mapa de flota. Leaflet con OpenStreetMap muestra la ubicación de buses en tiempo real (RF-5) sin costo de licenciamiento.
- Operación sin conexión. Con una PWA (service worker) e IndexedDB, la PC de cada estación guarda localmente conteos, transacciones e incidentes cuando se pierde la red y

los sincroniza al reconectar (RF-12). Cada registro lleva un identificador único para que el servidor evite duplicados, igual que la idempotencia aplicada en el motor de pagos de Hunahpú.

- Tailwind CSS. Interfaces consistentes y rápidas de construir, adecuadas al cronograma de 10 semanas.

3.6 IDE: Visual Studio Code

Se adopta Visual Studio Code por ser gratuito, tener soporte nativo de TypeScript y ofrecer extensiones para ESLint, Prettier, Docker, cliente REST y conexión a PostgreSQL. Además integra Git y el depurador de Node.js, lo que unifica el flujo de trabajo del equipo sin costos de licencia, algo relevante para un proyecto municipal.

3.7 Seguridad integral

- Autenticación JWT. Sesiones stateless con validación del usuario contra la base de datos en cada petición, para que los cambios de rol o la desactivación de una cuenta surtan efecto de inmediato.
- Control de acceso por roles y alcance. Además del rol (administrador, operador, guardia, piloto, personal municipal), se restringe por pertenencia: un operador solo opera sobre su estación y el personal municipal sobre su municipalidad (equivalente al control de propiedad de Hunahpú).
- Validación con Zod. Todo cuerpo, parámetro y consulta se valida antes de llegar a la lógica de negocio.
- Protección de datos. bcrypt para contraseñas, cifrado de datos personales de pilotos y de transacciones (RNF-3), TLS en Nginx para el tráfico y registro de auditoría. Esto responde a la recomendación de la factibilidad legal de adoptar buenas prácticas de protección de datos desde el diseño.

3.8 Tiempo real, pagos e integraciones

- Socket.IO. Transmite posiciones GPS/AVL y alertas de sobrecupo (ocupación igual o mayor al 150%) al centro de control en el momento en que ocurren (RNF-2, RF-7), y notifica de inmediato el botón de pánico (RF-8).
- Recaudo con tarjeta de acceso. El módulo de pagos se construye sobre una interfaz de adaptador, de modo que el proveedor final de validadores y pasarela pueda definirse en el proceso de contratación sin reescribir el sistema. Se reutilizan los patrones de Hunahpú: confirmaciones por webhook con firma verificada, conciliación automática e idempotencia para evitar cobros duplicados. Stripe queda como opción para recargas en línea, no como requisito.
- Cámaras IP y GPS/AVL. Se integran mediante la API REST documentada; el sistema registra eventos e incidentes y enlaza las grabaciones, sin depender de un fabricante específico.

3.9 DevOps, calidad y nube

- Docker Compose. Entorno local idéntico para todos los desarrolladores (PostgreSQL y pgAdmin), eliminando el problema de «funciona en mi máquina».
- Git y GitHub. Repositorio único y controlado, deficiencia señalada en la sección 4.4 del análisis de controles. Monorepositorio con frontend y backend separados.
- GitHub Actions. Ejecuta lint, pruebas y análisis de calidad en cada cambio.
- Pruebas. Jest y Supertest para pruebas funcionales y de integración de la API, Playwright para flujos de extremo a extremo, k6 para pruebas de carga y SonarQube para métricas de calidad del código.
- AWS EC2 con Nginx. Despliegue en la nube como se planteó en la factibilidad técnica; Nginx actúa como proxy inverso y terminación TLS. El API sin estado permite escalar horizontalmente (RNF-4), y los respaldos automáticos y reinicio automático de contenedores apoyan la disponibilidad 24 horas (RNF-1 y RNF-7). Una base de datos administrada (Amazon RDS) es una evolución posible para mayor alta disponibilidad.

4. Trazabilidad: requerimientos y tecnología

Requerimiento	Tecnología que lo atiende
RF-1, RF-2, RF-3, RF-4, RF-6	PostgreSQL (restricciones, triggers, transacciones), Sequelize y capa de Services.
RF-5 Geolocalización	Endpoint de ingesta GPS/AVL, Socket.IO y Leaflet; PostGIS opcional.
RF-7 Ocupación y alertas	Services de cálculo de ocupación y Socket.IO para la notificación en vivo.
RF-8 Cámaras y botón de pánico	Integración por API REST con cámaras IP y notificación por Socket.IO.
RF-9 Pago con tarjeta	Adaptador de pasarela, webhooks firmados, idempotencia y transacciones ACID.
RF-10, RF-11 Reportes y tableros	React, Recharts, vistas SQL y exportación XLSX/CSV.
RF-12 Sincronización local	PWA con service worker, IndexedDB y sincronización idempotente.
RNF-1 Disponibilidad 24 h	Docker, Nginx, reinicio automático y respaldos programados.
RNF-2 Tiempo real	Node.js asíncrono y Socket.IO.
RNF-3 Cifrado y roles	JWT, bcrypt, RBAC por rol y alcance, pgcrypto y TLS.
RNF-4 Crecimiento	API sin estado, despliegue en nube y esquema versionado con migraciones.

Requerimiento	Tecnología que lo atiende
RNF-5 Integración futura	API REST documentada con OpenAPI.
RNF-6 Historial de asignaciones	Tablas de auditoría con triggers o registro en Services.
RNF-7 Respaldo y recuperación	pg_dump programado, archivado de WAL e instantáneas del servidor.

5. Alternativas consideradas

Decisión	Alternativa	Motivo de descarte
Base de datos	MySQL	Es el motor del script actual y es viable, pero PostgreSQL ofrece triggers, tipos y extensiones más completas para reglas de negocio, cifrado y datos geográficos, y el equipo ya lo usa en Hunahpú.
Backend	Java Spring Boot o .NET	Son robustos, pero implican otra curva de aprendizaje y mayor tiempo de arranque en un cronograma de 10 semanas.
Frontend	Angular o Vue	Son válidos; React ya se domina y su ecosistema de mapas y gráficos es muy amplio.
Metodología	Cascada pura	Retrasa el software funcional y la detección de errores; no aprovecha la retroalimentación de los usuarios.

6. Conclusión

El stack propuesto es coherente con los requerimientos, con la arquitectura de componentes y con las conclusiones de los estudios de factibilidad. Se apoya en tecnologías maduras y de código abierto, ya conocidas por el equipo, lo que reduce el riesgo técnico y el costo de licenciamiento, y habilita desde el diseño el aseguramiento de la calidad, la seguridad y la escalabilidad que el Transmetro necesita.

1. Métricas de producto (calidad del código)

Se miden de forma automática con SonarQube en cada integración (GitHub Actions).

Métrica	Qué mide	Meta propuesta
Líneas de código (LOC / KLOC)	Tamaño del sistema, por módulo (Autenticación, Buses y Flota, Recaudo, etc.).	Solo referencia; sirve para normalizar otras métricas.
Complejidad ciclomática	Número de caminos lógicos por función.	≤ 10 por función
Complejidad cognitiva	Qué tan difícil es entender el código.	≤ 15 por función
Duplicación de código	Porcentaje de líneas repetidas.	< 3 %
Deuda técnica	Tiempo estimado para corregir problemas de mantenibilidad.	Ratio < 5 %
Índice de mantenibilidad	Calificación general de mantenibilidad.	A
Acoplamiento entre módulos	Dependencias entre los módulos de la API.	Bajo; la lógica de negocio solo en la capa Services.

Por qué importan aquí: la lógica más delicada (cálculo de ocupación, rango de buses por línea, descuento de saldo) vive en Services. Mantenerla simple facilita probarla.

2. Métricas de pruebas y defectos

Se miden con Jest, Supertest, Playwright y k6.

Métrica	Fórmula	Meta propuesta
Cobertura de código	Líneas ejecutadas por pruebas ÷ líneas totales × 100	≥ 80 % global y ≥ 90 % en Services críticos
Cobertura de requerimientos	RF y RNF con al menos un caso de prueba ÷ 19 × 100	100 % (12 RF + 7 RNF)
Tasa de éxito de pruebas	Pruebas aprobadas ÷ pruebas ejecutadas × 100	≥ 95 % antes de la entrega
Densidad de defectos	Defectos encontrados ÷ KLOC	< 1 defecto/KLOC en la entrega
Eficiencia de remoción de defectos (DRE)	Defectos hallados antes de la entrega ÷ (antes + después) × 100	≥ 90 %
Defectos críticos abiertos	Conteo de defectos de severidad alta sin resolver	0 en la aceptación final

Trazabilidad: con la cobertura de requerimientos se arma una matriz de trazabilidad (RF/RNF → casos de prueba), que además responde a la deficiencia 4.3 del análisis de controles.

3. Métricas de rendimiento, seguridad y disponibilidad

Salen directamente de los requerimientos no funcionales.

Métrica	RNF relacionado	Meta propuesta
Latencia de alertas y posiciones GPS (percentil 95)	RNF-2 tiempo real	≤ 2 segundos, a validar con k6
Tiempo de respuesta de la API (percentil 95)	RNF-2	≤ 500 ms en consultas normales
Usuarios y buses concurrentes soportados	RNF-4 crecimiento	Probar con 60 buses reportando y 25 estaciones conectadas
Disponibilidad	RNF-1 (24 h)	99.9 % (unas 8.76 horas de caída al año)
Vulnerabilidades críticas y altas	RNF-3	0 abiertas (revisión con SonarQube y npm audit)
RPO / RTO de respaldo	RNF-7	RPO ≤ 24 h, RTO ≤ 4 h
Sincronización sin pérdida de datos	RF-12	100 % de registros offline replicados sin duplicados

4. Métricas de proceso (Scrum)

Métrica	Qué mide	Uso
Velocidad del equipo	Historias completadas por sprint.	Predecir si se llega a los hitos de las semanas 4, 8 y 10.
Burndown del sprint	Trabajo pendiente frente al tiempo.	Detectar atrasos a mitad de sprint.
Cumplimiento de hitos	Entregables a tiempo ÷ entregables planificados.	Meta: 3 de 3 hitos de pago.
Tiempo de ciclo	Días desde que una historia inicia hasta que termina.	Detectar cuellos de botella.
Tasa de retrabajo	Historias reabiertas ÷ historias cerradas.	Meta: < 10 %.

5. Alineación con ISO/IEC 25010

Como estándar de referencia, este es el mapeo natural entre las características de calidad y las métricas definidas:

Característica de calidad	Métricas que la cubren
Adecuación funcional	Cobertura de requerimientos, tasa de éxito de pruebas.

Característica de calidad	Métricas que la cubren
Eficiencia de desempeño	Latencia, tiempo de respuesta, concurrencia.
Fiabilidad	Disponibilidad, RPO/RTO, sincronización offline.
Seguridad	Vulnerabilidades abiertas, control de acceso por roles.
Mantenibilidad	Complejidad, duplicación, deuda técnica, índice de mantenibilidad.
Compatibilidad	Cumplimiento del contrato OpenAPI para GPS/AVL, cámaras y validadores.

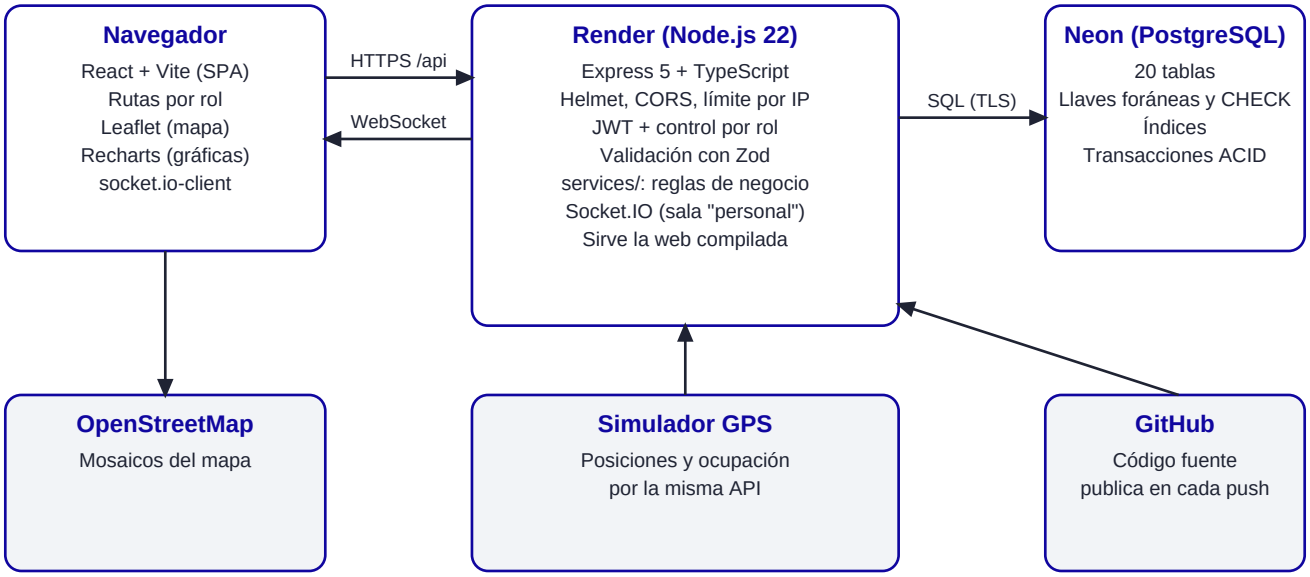
5

Implementación del sistema

Arquitectura, stack implementado frente al propuesto, estructura del repositorio, modelo de datos, reglas de negocio, seguridad, tiempo real y publicación en la nube.

5.1 Arquitectura

El sistema es una aplicación **cliente-servidor en tres capas**. El navegador ejecuta una aplicación de una sola página (SPA) en React; el servidor expone una API REST en Node.js con Express y un canal de tiempo real con Socket.IO; los datos viven en PostgreSQL. En producción un solo servicio entrega la página web y la API desde la misma dirección.



5.2 Stack implementado frente al propuesto

La sección 4 definió el stack objetivo. La tabla resume qué se implementó en esta entrega y qué cambió, con su motivo.

Componente	Propuesto (sección 4)	Implementado	Estado
Base de datos	PostgreSQL + Sequelize con migraciones	PostgreSQL 16 + Sequelize; tablas creadas con sync (sin migraciones versionadas)	**Parcial**
Backend	Node.js + Express 5, TypeScript	Igual, organizado en rutas, middlewares y services	**Completo**
Frontend	React + Tailwind CSS	React 18 + Vite + Tailwind + React Router	**Completo**
Tiempo real	Socket.IO	Socket.IO con autenticación por token y sala del personal	**Completo**
Seguridad	JWT, bcrypt, Zod, RBAC, TLS, cifrado de columnas	JWT, bcrypt, Zod, RBAC por rol y alcance, Helmet, límite por IP, AES-256-GCM para DPI y NIT; TLS lo da Render	**Parcial**
Mapas y reportes	Leaflet + OSM, Recharts, XLSX/CSV	Leaflet + OSM, Recharts, exportación CSV	**Completo**
Pruebas	Jest, Supertest, Playwright, k6, SonarQube	Jest + Supertest (103 casos) y colección de Postman; mediciones de rendimiento con script propio	**Parcial**
DevOps y nube	Docker Compose, GitHub, GitHub Actions, AWS EC2 + Nginx	Docker Compose (local), GitHub, Render + Neon	**Parcial**

Componente	Propuesto (sección 4)	Implementado	Estado
Documentación de API	Swagger / OpenAPI	Endpoints documentados en el README y en Postman	**Pendiente**
Operación sin conexión	PWA + IndexedDB	Referencia idempotente en cobros y recargas (base para sincronizar)	**Fase 2**

Cambio de nube. Se publicó en Render y Neon en lugar de AWS EC2 con Nginx porque ambos ofrecen un plan gratuito sin tarjeta de crédito, suficiente para la demostración y sin riesgo de cobro. La aplicación solo depende de variables de entorno, por lo que puede migrarse a AWS sin cambiar el código.

5.3 Estructura del repositorio

Carpeta o archivo	Contenido
backend/src/routes	Endpoints de la API y un CRUD genérico reutilizable para los catálogos
backend/src/services	Reglas de negocio: buses, líneas, pagos, capacidad, demanda, finanzas, horarios, reportes
backend/src/middleware	Autenticación JWT, control por rol y límite de intentos por IP
backend/src/db.ts	Modelos de Sequelize de las 20 tablas y sus relaciones
backend/tests	Pruebas unitarias e de integración, nombradas con el ID del plan (CP-xx)
frontend/src/pages	Pantallas: inicio público, tablero, mapa en vivo, alertas, líneas, buses, cobros, finanzas, demanda, reportes, etc.
database/	Scripts SQL de PostgreSQL: esquema, triggers, datos de demostración y consultas de los reportes (anexos A y B)
pruebas/	Colección y entorno de Postman
docs/	Guía técnica y guion de la presentación
DESPLIEGUE.md, render.yaml	Guía y configuración de la publicación en Render y Neon

5.4 Modelo de datos implementado

El modelo lógico del ER de la sección 2 se mantiene. Para PostgreSQL se usaron columnas SERIAL y TIMESTAMPTZ, como indicó la nota de adaptación de la sección 4, y se agregaron las tablas y columnas que pedían los requerimientos:

- **usuario**: cuentas con rol, estación o municipalidad asignada, y DPI y NIT cifrados (RNF-3).
- **auditoria_bus**: historial de cambios de línea y parqueo de cada bus, con fecha y usuario (RNF-6).
- **latitud y longitud** en estación y bus, y **ultima_posicion** en bus (RF-5).
- **referencia** única en transacción, para que un reintento no cobre dos veces (RF-9 y base de RF-12).
- **atendida** en alerta; tablas de **horarios**, **sugerencias** y **movimiento_financiero**.
- **bus.id_parqueo NOT NULL**: el script original lo permitía nulo, pero RF-4 prohíbe que un bus quede sin parqueo.
- Restricciones CHECK: saldo mayor o igual a cero, montos y capacidades positivas, roles válidos y horas de inicio menores que las de fin.

5.5 Reglas de negocio y su verificación

Req.	Regla	Dónde se aplica	Casos
RF-1	Distancia total de la línea = suma de sus tramos; distancia entre estaciones	services/lineas.ts	CP-10 a CP-15
RF-2	Al menos un guardia por acceso (el reporte marca la cobertura)	services/reportes.ts	CP-16, CP-17
RF-3	Cada línea con N a 2N buses, validado en transacción con bloqueo de la línea	services/buses.ts	CP-19 a CP-23
RF-4	Un bus nunca queda sin parqueo	db.ts y services/buses.ts	CP-24 a CP-26
RF-7	Sobrecupo con espera de 150 % o más; bus bajo 25 % espera 5 min más	services/capacidad.ts	CP-29 a CP-33
RF-9	Cobro atómico (ACID) e idempotente; saldo nunca negativo	services/pagos.ts	CP-39 a CP-46
RNF-6	Cada cambio de línea o parqueo queda en el historial	services/buses.ts	CP-28

5.6 Seguridad

- Contraseñas con **bcrypt** (10 rondas con sal); nunca en texto plano.
- Sesión con **JWT** de 12 horas; el usuario se recarga en cada petición, así una cuenta desactivada pierde acceso de inmediato.
- Control por **rol y alcance**: el operador solo opera su estación y el personal municipal solo ve su municipalidad.
- Validación de toda entrada con **Zod**; consultas parametrizadas contra inyección SQL.
- **Helmet** con Content-Security-Policy; **límite de intentos por IP** en inicio de sesión, registro, recarga pública y sugerencias.
- DPI y NIT cifrados con **AES-256-GCM** y huella HMAC para detectar duplicados; en pantalla se muestran enmascarados.
- Los secretos viven en variables de entorno; en producción el servidor no arranca sin ellos.

5.7 Tiempo real y mapa

Cada bus envía su posición a `POST /api/buses/:id/posicion`. El servidor la guarda y la emite con el evento `bus:posicion` por Socket.IO solo al personal; el mapa en Leaflet mueve el marcador sin recargar. Las alertas, los incidentes y el botón de pánico usan el mismo canal. En la demostración, un simulador genera las posiciones usando la misma API que usaría un GPS real.

5.8 Publicación

En cada cambio a la rama principal de GitHub, Render descarga el código, compila el frontend, instala el backend y arranca el servidor, que crea las tablas que falten en Neon y, si la base está vacía, carga los datos de demostración. La conexión a la base viaja cifrada (sslmode=require) y el tráfico del sitio usa HTTPS.

6

Plan de pruebas, criterios y carta de aceptación

Parte A: plan de pruebas con 60 casos y matriz de trazabilidad. Parte B: criterios de aceptación funcionales, no funcionales y de entrega. Parte C: carta de aceptación.

Documento entregado el 21-08-2026. Los resultados de su ejecución están en la sección 7.

Transmetro: plan de pruebas, criterios de aceptación y carta de aceptación

Parte A. Plan de pruebas

A.1 Objetivo

Verificar que el sistema cumple los 12 requerimientos funcionales (RF-1 a RF-12) y los 7 no funcionales (RNF-1 a RNF-7) del análisis de requerimientos, que las reglas de negocio se aplican siempre en el backend y que el sistema es seguro y estable para su aceptación por las municipalidades.

A.2 Alcance

Se prueba: autenticación y roles, líneas y estaciones, accesos y guardias, buses, parqueos y pilotos, ocupación y alertas, posición GPS, incidentes y botón de pánico, cobros y recargas, portal del pasajero, reportes y tablero, tiempo real y rendimiento.

No se prueba en esta entrega (porque no está construido o depende de hardware o proveedores externos):

- Dispositivos GPS/AVL físicos: se prueba el endpoint que recibe la posición con un simulador.
- Cámaras IP y validadores físicos de tarjeta.
- Pasarela de pago real: la actual es una simulación.
- Almacenamiento y sincronización local sin conexión (RF-12). Solo se prueba la idempotencia del cobro, que es su base.

A.3 Estrategia y niveles de prueba

Nivel	Qué verifica	Herramienta	Estado
Unitarias	Reglas puras: rango de buses, umbrales de ocupación, distancias, CSV	Jest (carpeta tests/unit)	Escritas; por ejecutar
Integración de API	Endpoints, códigos de estado, reglas y permisos por rol	Supertest (tests/integration) y Thunder Client (colección lista)	Escritas; por ejecutar
Funcionales de interfaz	Flujos de cada rol en la web	Manual con lista de casos	Por ejecutar
Extremo a extremo	Flujos completos de navegador a base de datos	Playwright	Por implementar
Carga y rendimiento	Latencia, concurrencia	k6	Por implementar
Seguridad	Acceso no autorizado, datos inválidos, dependencias	Casos manuales y npm audit	Por ejecutar
Aceptación (UAT)	Cumplimiento de los criterios de la Parte B	Demostración guiada	Por ejecutar

Se aplica caja negra (entradas y salidas) para la API y la interfaz, y caja blanca (lectura del código) en las reglas de `services/`. Las pruebas de regresión se repiten completas antes de la entrega. Los casos automatizados llevan en su nombre el mismo identificador de este plan (CP-01, CP-02, etc.).

A.4 Entorno y datos de prueba

- PostgreSQL 16 en Docker, backend en Node.js (puerto 4000) y frontend en Vite (puerto 5173), todo local. Las pruebas automáticas usan una base aparte llamada transmetro_test.
- Datos de partida: `npm run seed` (3 municipalidades, 8 estaciones, 3 líneas, 14 buses, 8 pilotos, 3 tarjetas) o los scripts de la carpeta `database/`. Antes de cada ciclo se reinician los datos para que los resultados sean repetibles.
- Usuarios por rol: administrador, personal municipal, operador de estación, guardia y pasajero.
- Los identificadores de los casos (bus 13, línea 1, etc.) corresponden a los datos del seed.

A.5 Criterios de entrada, salida y suspensión

- **Entrada:** el backend y el frontend arrancan sin errores, la base está cargada con el seed y existe la colección de pruebas.
- **Salida:** todos los casos de prioridad Alta ejecutados, tasa de éxito de al menos 95 %, cero defectos críticos abiertos y los criterios de la Parte B cumplidos.
- **Suspensión:** si el sistema no arranca, la base no responde o un defecto crítico impide probar el resto, se detiene el ciclo hasta corregirlo.

A.6 Casos de prueba

Prioridad: A = alta, M = media. La columna Estado empieza en Pendiente y se actualiza al ejecutar (Aprobado, Fallido o Bloqueado), anotando fecha y evidencia (captura o respuesta de la API).

Autenticación, roles y seguridad

ID	Req.	Caso	Datos y pasos	Resultado esperado	Prio.	Estado
CP-01	RNF-3	Inicio de sesión correcto	Correo y contraseña válidos del administrador	200 y un token	A	Pendiente
CP-02	RNF-3	Contraseña incorrecta	Correo válido, clave errada	401 con mensaje genérico que no revela si el correo existe	A	Pendiente
CP-03	RNF-3	Petición sin token	GET a lineas sin cabecera	401	A	Pendiente
CP-04	RNF-3	Rol sin permiso	Guardia intenta crear una línea	403	A	Pendiente
CP-05	RNF-3	Alcance por estación	Operador de Centra Norte registra ocupación con id_estacion de otra estación	403	A	Pendiente
CP-06	RNF-3	Pasajero en rutas del personal	Pasajero consulta pilotos	403	A	Pendiente
CP-07	RNF-3	Contraseñas protegidas	Revisar la tabla usuario en la base	Solo hay hashes bcrypt, no texto plano	A	Pendiente
CP-08	RNF-3	Cuenta desactivada	Desactivar un usuario con sesión abierta y repetir una petición	401 inmediato	M	Pendiente
CP-09	RNF-3	Validación de datos	Correo inválido, monto negativo, latitud 200	400 con la lista de campos inválidos	A	Pendiente

Líneas, estaciones, accesos y guardias (RF-1, RF-2)

ID	Req.	Caso	Datos y pasos	Resultado esperado	Prio.	Estado
CP-10	RF-1	Estación en varias líneas	Trébol está en las líneas 1, 2 y 3	Aparece en las tres líneas y en el reporte de estaciones	A	Pendiente
CP-11	RF-1	Recorrido ordenado y distancia total	Definir la línea 3 con tramos de 6.0 y 4.9 km	Orden 1, 2, 3 y distancia total 10.9 km calculada sola	A	Pendiente
CP-12	RF-1	Estación repetida en una línea	Enviar la misma estación dos veces	400	M	Pendiente
CP-13	RF-1	Distancia entre estaciones de una línea	De Centra Norte a Plaza Barrios en la línea 1	4.8 km (1.7 + 3.1)	A	Pendiente
CP-14	RF-1	Distancia entre estaciones sin línea común	De Centra Norte a Villa Nueva Centro	Distancia en línea recta por coordenadas	M	Pendiente
CP-15	RF-1	Municipalidad de línea y estación	Crear línea y estación sin municipalidad	400	M	Pendiente
CP-16	RF-2	Acceso con guardia	Crear acceso y asignarle un guardia	Reporte de estaciones marca cobertura completa	A	Pendiente
CP-17	RF-2	Acceso sin guardia	Crear un acceso sin guardia	El reporte muestra cobertura incompleta y nombra el acceso	A	Pendiente
CP-18	RF-2	Operador por estación	Crear operador ligado a una estación	Queda registrado y su usuario solo opera en esa estación	M	Pendiente

Buses, parqueos y pilotos (RF-3, RF-4, RF-6, RNF-6)

ID	Req.	Caso	Datos y pasos	Resultado esperado	Prio.	Estado
CP-19	RF-3	Asignar bus dentro del rango	Bus 13 a la línea 1 (4 estaciones, hoy 5 buses)	200 y el bus queda en la línea	A	Pendiente
CP-20	RF-3	Superar el máximo	Asignar buses hasta tener 9 en la línea 1	409 al intentar el noveno (máximo 2 x 4 = 8)	A	Pendiente
CP-21	RF-3	Bajar del mínimo	Retirar un bus de la línea 3 (3 buses, 3 estaciones)	409	A	Pendiente
CP-22	RF-3	Línea sin estaciones	Crear línea nueva y asignarle un bus	409	A	Pendiente
CP-23	RF-3	Un bus en una sola línea	Pasar un bus de la línea 1 a la 2	Deja de aparecer en la 1	A	Pendiente
CP-24	RF-4	Bus sin parqueo	Crear bus sin id_parqueo	400	A	Pendiente
CP-25	RF-4	Cambio de parqueo	Cambiar un bus a otro parqueo; luego enviar parqueo vacío	200 y luego 400; nunca queda sin parqueo	A	Pendiente
CP-26	RF-4	Capacidad máxima	Crear bus con capacidad 0	400	M	Pendiente
CP-27	RF-6	Registro de piloto	Nombre, historial educativo, residencia, teléfono, correo y bus	Queda asociado al bus y a su línea	A	Pendiente
CP-28	RNF-6	Historial de asignaciones	Consultar el historial de un bus con cambios	Lista fecha, línea y parqueo anterior y nuevo	A	Pendiente

Ocupación, GPS e incidentes (RF-5, RF-7, RF-8)

ID	Req.	Caso	Datos y pasos	Resultado esperado	Prio.	Estado
CP-29	RF-7	Sobrecupo	Bus de 120: 180 en espera (150 %)	Alerta SOBRECUPO y aviso en vivo	A	Pendiente
CP-30	RF-7	Límite inferior del sobrecupo	Bus de 120: 179 en espera	Sin alerta	A	Pendiente
CP-31	RF-7	Poca ocupación	Bus de 120: 29 a bordo (24 %)	Espera adicional de 5 minutos y alerta	A	Pendiente
CP-32	RF-7	Límite del 25 %	Bus de 120: 30 a bordo	Sin espera adicional	A	Pendiente
CP-33	RF-7	Marcar alerta atendida	Operador atiende una alerta	Cambia a atendida y sale del tablero	M	Pendiente
CP-34	RF-5	Posición GPS	Enviar latitud y longitud del bus 1	Se guarda y el punto se mueve en el mapa sin recargar	A	Pendiente
CP-35	RF-5	Coordenadas inválidas	Latitud 200	400	M	Pendiente
CP-36	RF-8	Botón de pánico	Guardia lo activa	Se registra el incidente y el centro de control recibe aviso inmediato	A	Pendiente
CP-37	RF-8	Incidente de seguridad	Registrar un robo en una estación	Aparece en la lista con fecha y estación	M	Pendiente
CP-38	RF-5, RF-8	Pasajero no recibe alertas internas	Sesión de pasajero abierta y se dispara una alerta	El pasajero no recibe el aviso	A	Pendiente

Pagos y portal del pasajero (RF-9)

ID	Req.	Caso	Datos y pasos	Resultado esperado	Prio.	Estado
CP-39	RF-9	Cobro de pasaje	Tarjeta con Q25 y tarifa base	Saldo baja Q1 y queda una transacción con estación y hora	A	Pendiente
CP-40	RF-9	Saldo insuficiente	Tarjeta con Q0.50	409 y no se crea transacción	A	Pendiente
CP-41	RF-9	Cobro repetido	Enviar dos veces la misma referencia	La segunda respuesta indica duplicada y el saldo no baja otra vez	A	Pendiente
CP-42	RF-9	Cobros simultáneos	Dos cobros a la vez con saldo para uno solo	Solo uno se aprueba; el saldo nunca queda negativo	A	Pendiente
CP-43	RF-9	Recarga en estación	Operador recarga Q10	Saldo sube y queda como recarga	M	Pendiente
CP-44	RF-9	Registro y tarjeta del pasajero	Crear cuenta y solicitar tarjeta	Se crea cuenta de pasajero con una tarjeta nueva a su nombre	A	Pendiente
CP-45	RF-9	Recarga en línea	Pasajero recarga Q20	Saldo sube y el movimiento figura sin estación	A	Pendiente
CP-46	RF-9	Historial propio y ajeno	Pasajero consulta su tarjeta y luego una ajena	Ve su historial con último uso; la ajena responde 404	A	Pendiente

Reportes y tablero (RF-10, RF-11)

ID	Req.	Caso	Datos y pasos	Resultado esperado	Prio.	Estado
CP-47	RF-10	Reporte de estaciones	Abrir el reporte	Lista líneas, accesos, guardias y cobertura por estación	A	Pendiente
CP-48	RF-10	Reporte de líneas y buses	Abrir el reporte	Cada línea con sus buses, rango permitido y si cumple	A	Pendiente
CP-49	RF-10	Exportar a CSV	Descargar los tres reportes	Se abren en Excel con acentos correctos	M	Pendiente
CP-50	RF-11	Tablero	Abrir el tablero	Cifras iguales a las de la base de datos y gráfica por línea	A	Pendiente
CP-51	RF-10	Reporte limitado por municipalidad	Personal municipal abre los reportes	Solo ve su municipalidad	M	Pendiente

No funcionales e interfaz

ID	Req.	Caso	Datos y pasos	Resultado esperado	Prio.	Estado
CP-52	RNF-2	Latencia de alertas y posiciones	k6 con 60 buses enviando posición	Percentil 95 de 2 segundos o menos	A	Pendiente
CP-53	RNF-2	Tiempo de respuesta de la API	k6 en consultas normales	Percentil 95 de 500 ms o menos	A	Pendiente
CP-54	RNF-4	Concurrencia y crecimiento	60 buses y 25 estaciones conectados; luego 100 líneas cargadas	Sin errores ni degradación notable	M	Pendiente
CP-55	RNF-7	Respaldo y recuperación	Respaldo con pg_dump y restauración en otra base	Datos idénticos; RPO de 24 h y RTO de 4 h o menos	A	Pendiente
CP-56	RNF-1	Reinicio automático	Detener el contenedor de la base y levantarlo	El sistema se recupera sin perder datos	M	Pendiente
CP-57	RNF-3	Vulnerabilidades	npm audit del backend y el frontend	Cero críticas y altas abiertas	A	Pendiente
CP-58	RNF-5	Integración futura	Revisar el contrato de la API	Endpoints consistentes y documentados	M	Pendiente
CP-59	UI	Menú por rol	Entrar con cada rol	Cada uno ve solo sus secciones; el menú abre y cierra con botón y con Escape	M	Pendiente
CP-60	UI	Mensajes de error	Provocar una regla de negocio desde la pantalla	Se muestra el motivo en lenguaje claro	M	Pendiente

A.7 Matriz de trazabilidad

Requerimiento	Casos de prueba
RF-1	CP-10 a CP-15
RF-2	CP-16 a CP-18
RF-3	CP-19 a CP-23
RF-4	CP-24 a CP-26
RF-5	CP-34, CP-35, CP-38
RF-6	CP-27
RF-7	CP-29 a CP-33
RF-8	CP-36, CP-37, CP-38
RF-9	CP-39 a CP-46
RF-10	CP-47 a CP-49, CP-51
RF-11	CP-50
RF-12	CP-41 (solo la idempotencia; el resto queda para la fase 2)
RNF-1	CP-56
RNF-2	CP-52, CP-53
RNF-3	CP-01 a CP-09, CP-57
RNF-4	CP-54
RNF-5	CP-58
RNF-6	CP-28
RNF-7	CP-55

Así se calcula la cobertura de requerimientos (meta del 100 %) y se responde a la deficiencia de trazabilidad señalada en el análisis de controles.

A.8 Gestión de defectos

Severidad	Significado	Ejemplo	Plazo de corrección
Crítica	El sistema no funciona o hay pérdida de datos o fallo de seguridad	Un pasajero lee datos de pilotos	Antes de continuar las pruebas
Alta	Una regla de negocio falla	Se permite pasar de 2N buses	Antes de la entrega
Media	Falla una función con alternativa	CSV sin acentos	Antes de la entrega si hay tiempo
Baja	Detalle visual o de texto	Alineación de una tabla	Siguiente versión

Cada defecto se registra con: identificador, caso de prueba, pasos para reproducirlo, resultado esperado y obtenido, severidad, evidencia, responsable y estado (Abierto, En corrección, Corregido, Verificado, Cerrado). Un defecto corregido se vuelve a probar y se repiten los casos relacionados. Con estos registros se calculan la densidad de defectos y la eficiencia de remoción (meta de 90 % o más).

A.9 Cronograma

Semana	Actividad
7	Diseño de casos, preparación del entorno y datos, pruebas unitarias de los servicios
8	Pruebas de integración de API y funcionales de interfaz
9	Pruebas de seguridad, carga y respaldo; corrección de hallazgos y regresión
10	Pruebas de aceptación con las municipalidades y firma de la carta

A.10 Riesgos de las pruebas

- Pruebas de carga y de seguridad que requieren un entorno parecido al real: se mitiga ejecutándolas en el mismo despliegue que se entrega.
- Datos de prueba alterados entre ciclos: se mitiga reiniciando con el seed antes de cada ciclo.
- Integraciones externas simuladas (GPS, pasarela): se declaran como tales y se prueban a nivel de interfaz.
- Poco tiempo para correcciones: se prioriza por severidad.

A.11 Registro de ejecución (para llenar)

Ciclo	Fecha	Casos ejecutados	Aprobados	Fallidos	Bloqueados	% de éxito	Defectos nuevos
1							
2 (regresión)							
Final							

Parte B. Criterios de aceptación

El sistema se acepta cuando cumple los criterios de esta parte, verificados con los casos de la Parte A. La columna Alcance indica lo que cubre esta entrega: **Completo**, **Parcial** (la parte de software funciona y el hardware o proveedor externo queda por integrar) o **Fase 2**.

B.1 Criterios funcionales

Req.	Criterio de aceptación	Alcance	Casos
RF-1	Dado un administrador, cuando define el recorrido de una línea, entonces las estaciones quedan en orden con la distancia de cada tramo y la distancia total es su suma; una estación puede estar en varias líneas y cada línea y estación pertenece a una municipalidad	Completo	CP-10 a CP-15
RF-2	Dada una estación, cuando se registran accesos, entonces cada uno acepta guardias y el reporte señala los accesos sin guardia; cada estación tiene un operador con acceso desde la web	Completo	CP-16 a CP-18
RF-3	Dado un bus, cuando se asigna a una línea, entonces pertenece solo a esa línea y el sistema rechaza superar el doble de las estaciones o dejar la línea por debajo del número de estaciones	Completo	CP-19 a CP-23
RF-4	Dado un bus, cuando se registra o se cambia de parqueo, entonces siempre conserva uno asignado y su capacidad máxima es mayor que cero	Completo	CP-24 a CP-26
RF-5	Dada la posición de un bus, cuando llega al sistema, entonces se guarda y se ve moverse en el mapa en menos de 2 segundos	Parcial: endpoint y mapa listos; dispositivo GPS por integrar	CP-34, CP-35
RF-6	Dado un piloto, cuando se registra, entonces quedan su historial educativo, residencia y contacto asociados a su bus y su línea	Completo	CP-27
RF-7	Dados pasajeros en espera iguales o mayores al 150 % de la capacidad del bus, entonces se genera alerta; con menos del 25 % a bordo, el bus espera 5 minutos más	Completo con datos ingresados por API; conteo automático por sensores es Fase 2	CP-29 a CP-33
RF-8	Dado un guardia u operador, cuando activa el botón de pánico, entonces el incidente se registra y el centro de control recibe aviso inmediato; los incidentes se pueden registrar y consultar	Parcial: pánico e incidentes listos; cámaras IP por integrar	CP-36 a CP-38
RF-9	Dada una tarjeta de acceso, cuando se cobra o se recarga, entonces el saldo y la transacción cambian juntos, no hay saldo negativo ni cobros duplicados y el pasajero ve su saldo e historial	Parcial: cobro, recarga y portal del pasajero listos; pasarela real y validadores físicos por integrar	CP-39 a CP-46
RF-10	Dados los reportes de estaciones, líneas con buses y accesos por línea, cuando se abren o exportan, entonces muestran datos correctos y se descargan en CSV	Completo	CP-47 a CP-49, CP-51
RF-11	Dado el tablero, cuando se abre, entonces muestra las cifras de la red y los puntos que requieren atención	Completo	CP-50

RF-12	Dada una estación sin conexión, cuando vuelve la red, entonces sus registros se sincronizan sin duplicados	Fase 2 (la idempotencia de cobros ya existe)	CP-41

B.2 Criterios no funcionales

Req.	Criterio medible	Alcance	Casos
RNF-1	Disponibilidad objetivo de 99.9 % con reinicio automático de contenedores	Parcial: depende del despliegue en la nube	CP-56
RNF-2	Alertas y posiciones con percentil 95 de 2 s o menos; API con percentil 95 de 500 ms o menos	Completo (a validar con k6)	CP-52, CP-53
RNF-3	Contraseñas con hash, acceso por roles y alcance, validación de datos y cero vulnerabilidades críticas o altas	Parcial: cifrado en reposo de datos de pilotos y TLS pendientes	CP-01 a CP-09, CP-57
RNF-4	Funciona con 60 buses y 25 estaciones y admite más líneas sin cambiar el diseño	Completo	CP-54
RNF-5	API REST con contrato consistente para integrar otros sistemas	Parcial: documentación Swagger pendiente	CP-58
RNF-6	Todo cambio de línea o parqueo de un bus queda en el historial con fecha y usuario	Completo	CP-28
RNF-7	Respaldo y restauración comprobados con RPO de 24 h o menos y RTO de 4 h o menos	Pendiente de configurar y probar	CP-55

B.3 Criterios de calidad de la entrega

Métrica	Meta
Cobertura de requerimientos con al menos un caso	100 %
Tasa de éxito de las pruebas	95 % o más antes de la entrega
Defectos críticos abiertos	0
Eficiencia de remoción de defectos	90 % o más
Densidad de defectos	Menos de 1 por cada mil líneas de código
Cobertura de código en los servicios	80 % o más global y 90 % en los servicios críticos
Complejidad ciclomática por función	10 o menos
Duplicación de código	Menos del 3 %

B.4 Criterios sobre los entregables

- Documento PDF unificado con todos los puntos solicitados y los enlaces a los repositorios.
- Presentación con el resumen ejecutivo.
- Video de explicación general de al menos 10 minutos.
- Repositorio con el código, los scripts de base de datos y la documentación.
- Aplicación web funcional alojada en la nube, con los reportes visibles en la aplicación y los queries documentados.

B.5 Condición de aceptación final

Se acepta el proyecto cuando: (1) todos los casos de prioridad Alta están aprobados, (2) se cumplen las metas de la sección B.3, (3) los entregables de B.4 están completos y (4) las partes marcadas como Parcial o Fase 2 están reconocidas por escrito en la carta de aceptación como alcance pendiente. La aceptación se hace en una demostración guiada ante la municipalidad, que recorre los criterios de B.1 y B.2 con datos reales de prueba.

Parte C. Carta de aceptación del proyecto

Los espacios con línea se llenan al momento de la aceptación.

Guatemala, _____de _____de 2026

Señores
Equipo consultor Sistemas Especialidades
Atención: Ernesto Leonardo Oliva Esquivel
Presente

Referencia: Aceptación del Sistema de Control de Líneas, Estaciones y Buses del Transmetro Estimados señores:

Por este medio, nosotros, representantes de la(s) municipalidad(es) de _____, hacemos constar que hemos recibido y revisado los entregables del proyecto Sistema de Control de Líneas, Estaciones y Buses del Transmetro, desarrollado por el equipo consultor, y que hemos presenciado la ejecución de las pruebas de aceptación conforme al Plan de Pruebas y a los Criterios de Aceptación acordados.

1. Entregables recibidos

Entregable	Recibido (Sí / No)	Observaciones
Documento PDF unificado		
Presentación con resumen ejecutivo		
Video de explicación general		
Aplicación web funcional en la nube		
Repositorio de código y base de datos		
Scripts de creación de la base de datos		
Reportes de estaciones, líneas y buses asignados		
Plan de pruebas y registro de ejecución		

2. Resultado de las pruebas de aceptación

- Casos ejecutados: _____Aprobados: _____Fallidos: _____
- Tasa de éxito: _____% Defectos críticos abiertos: _____
- Requerimientos funcionales cumplidos: _____de 12 No funcionales cumplidos de 7

3. Alcance pendiente reconocido

Las partes siguientes quedan fuera de esta entrega y se reconocen como alcance pendiente: integración de dispositivos GPS/AVL, cámaras IP y validadores físicos de tarjeta; pasarela de pago real; almacenamiento y sincronización sin conexión; y los puntos de la sección B.2 marcados como Parcial. Otros: _____.

4. Decisión

( ) Aceptamos el proyecto a entera satisfacción.

( ) Aceptamos el proyecto con las siguientes observaciones, que el consultor corregirá en un plazo de _____días: _____.

( ) No aceptamos el proyecto por los siguientes motivos: _____.

5. Garantía, soporte y pago

A partir de la fecha de esta carta corre la garantía de 3 meses incluida en la propuesta y el soporte y mantenimiento de 12 meses. Con la aceptación final procede el último pago por hitos, equivalente al 25 % de la inversión total estimada de Q 1,314,000.00, es decir Q 328,500.00, conforme al calendario de pagos de la propuesta monetaria.

Sin otro particular, firmamos de conformidad.

Por la municipalidad	Por el equipo consultor
Nombre: _____	Nombre: Ernesto Leonardo Oliva Esquivel
Cargo: _____	Cargo: Consultor, equipo Sistemas Especialidades
Firma: _____	Firma: _____
Sello: _____	Fecha: _____

Testigo 1	Testigo 2
Nombre: _____	Nombre: _____
Firma: _____	Firma: _____

7

Resultados de la ejecución de pruebas

Registro de ejecución, resultado por caso, métricas de calidad obtenidas frente a las metas y registro de defectos.

7.1 Registro de ejecución

Se ejecutó un ciclo completo el 24-10-2026 sobre la versión final del código, con PostgreSQL 16 y los datos de demostración. Este registro completa la sección A.11 del plan de pruebas (sección 6).

Ciclo	Fecha	Casos ejecutados	Aprobados	Fallidos	Bloqueados	% de éxito	Defectos nuevos
1	24-10-2026	55 de 60	54	1	0	98.2 %	7
2 (regresión)	24-10-2026	55 de 60	54	1	0	98.2 %	0

Las pruebas automatizadas (Jest + Supertest) suman **103 casos** en 10 archivos, con **103 aprobados**. Cubren 48 de los 60 casos del plan; varios casos tienen más de una prueba automática. Los casos CP-28, CP-38, CP-52, CP-53, CP-55, CP-56 y CP-57 se ejecutaron con scripts y comandos contra el servidor local.

7.2 Resultado por caso de prueba

Caso	Resultado	Evidencia u observación
CP-01 a CP-09	**Aprobado**	Autenticación, roles y seguridad: prueba automática aprobada en todos los casos
CP-10 a CP-17	**Aprobado**	Líneas, estaciones, accesos y guardias: prueba automática aprobada en todos los casos
CP-18	**Pendiente**	Sin prueba automática; verificación manual pendiente
CP-19 a CP-27	**Aprobado**	Buses, parqueos y pilotos: prueba automática aprobada en todos los casos
CP-28	**Aprobado**	Bus 1 de la línea 1 a la 2: historial con línea anterior y nueva, fecha y usuario
CP-29 a CP-37	**Aprobado**	Ocupación, GPS e incidentes: prueba automática aprobada en todos los casos
CP-38	**Aprobado**	Pasajero conectado por Socket.IO: 0 eventos recibidos de posición ni de alerta
CP-39 a CP-46	**Aprobado**	Pagos y portal del pasajero: prueba automática aprobada en todos los casos
CP-47 a CP-51	**Aprobado**	Reportes y tablero: prueba automática aprobada en todos los casos
CP-52	**Aprobado**	Posición a evento en vivo, 60 muestras: p95 de 7 ms, máximo 12 ms (script propio, no k6)
CP-53	**Aprobado**	300 consultas a 6 rutas: p95 de 284 ms, máximo 371 ms
CP-54	**Pendiente**	Requiere k6 con 60 buses y 25 estaciones
CP-55	**Aprobado**	pg_dump y pg_restore en otra base: conteos y saldo total idénticos; 0.6 s
CP-56	**Aprobado parcial**	Con la base detenida la API responde 500; al reiniciarla vuelve a 200 sin reiniciar la app ni perder datos. El reinicio automático depende de Render y Neon
CP-57	**Fallido**	npm audit: backend con 1 alta (zod) y 2 moderadas (uuid); frontend con 2 moderadas (react-router)
CP-58	**Pendiente**	Contrato documentado en README y Postman; falta OpenAPI
CP-59	**Pendiente**	Evidencia parcial en la sección 8 (menús de administrador y pasajero)
CP-60	**Pendiente**	Verificación manual pendiente

7.3 Métricas de calidad obtenidas

Métrica	Meta (sección 4 y B.3)	Obtenido	Cumple
Tasa de éxito de las pruebas	95 % o más	98.2 % de los casos del plan; 100 % de las automáticas	Sí
Cobertura de requerimientos	100 % (19 requerimientos)	17 de 19 (89 %); faltan RNF-4 (CP-54) y RNF-5 (CP-58)	No
Cobertura de código global	80 % o más	93.7 % líneas, 88.1 % sentencias, 71.0 % ramas	Sí
Cobertura en servicios críticos	90 % o más	buses, capacidad y pagos: 100 % de líneas; líneas 98 %	Sí
Defectos críticos abiertos	0	0	Sí
Vulnerabilidades críticas y altas	0 abiertas	1 alta abierta (zod, requiere migrar a la versión 4)	No
Densidad de defectos	Menos de 1 por KLOC	7 defectos en unas 4.8 KLOC de TypeScript: 1.5 por KLOC	No
Latencia de tiempo real (p95)	2 s o menos	7 ms (local)	Sí
Tiempo de respuesta de la API (p95)	500 ms o menos	284 ms (local)	Sí
Respaldo: RPO y RTO	RPO 24 h o menos; RTO 4 h o menos	Restauración en menos de 1 s; respaldo programado pendiente de configurar	Parcial

Las mediciones de rendimiento se hicieron en un entorno local con datos de demostración (66,381 transacciones), sin la latencia de internet. Deben repetirse con k6 contra el sitio publicado para validar RNF-2 en condiciones reales.

7.4 Registro de defectos

Defectos encontrados y gestionados según la sección A.8 del plan.

ID	Descripción	Severidad	Estado	Corrección
D-01	El mapa en vivo mostraba el error "Cannot read properties of undefined (reading appendChild)"	Media	Cerrado	Se evita dibujar sobre un mapa ya destruido
D-02	En el sitio publicado los mosaicos del mapa salían como "Access blocked" (403): Helmet enviaba Referrer-Policy no-referrer y OpenStreetMap exige el Referer	Alta	Cerrado	Política strict-origin-when-cross-origin (commit 5e43ac7)
D-03	Las cuentas del personal no se creaban si la base ya tenía algún usuario, y el administrador no podía entrar	Alta	Cerrado	Al arrancar se crean las cuentas que falten (commit 5e43ac7)
D-04	El simulador usaba una contraseña fija y fallaba contra el sitio publicado	Baja	Cerrado	Credenciales por variables de entorno (commit f881f1f)
D-05	La prueba CP-44 no enviaba el teléfono y el DPI que ahora exige la primera solicitud de tarjeta	Media	Cerrado	Prueba actualizada (commit 2a12c5a)
D-06	Vulnerabilidad alta en body-parser 2.3.0 (GHSA-27r3-jxmh-p828)	Alta	Cerrado	Actualizado a 2.4.0 (commit 2a12c5a)

ID	Descripción	Severidad	Estado	Corrección
D-07	Vulnerabilidad alta en zod anterior a 4.6.6	Alta	**Abierto**	Requiere migrar a zod 4 (cambio mayor)
D-08	Vulnerabilidades moderadas en uuid (dependencia de Sequelize) y react-router	Media	**Abierto**	Actualización mayor pendiente

8

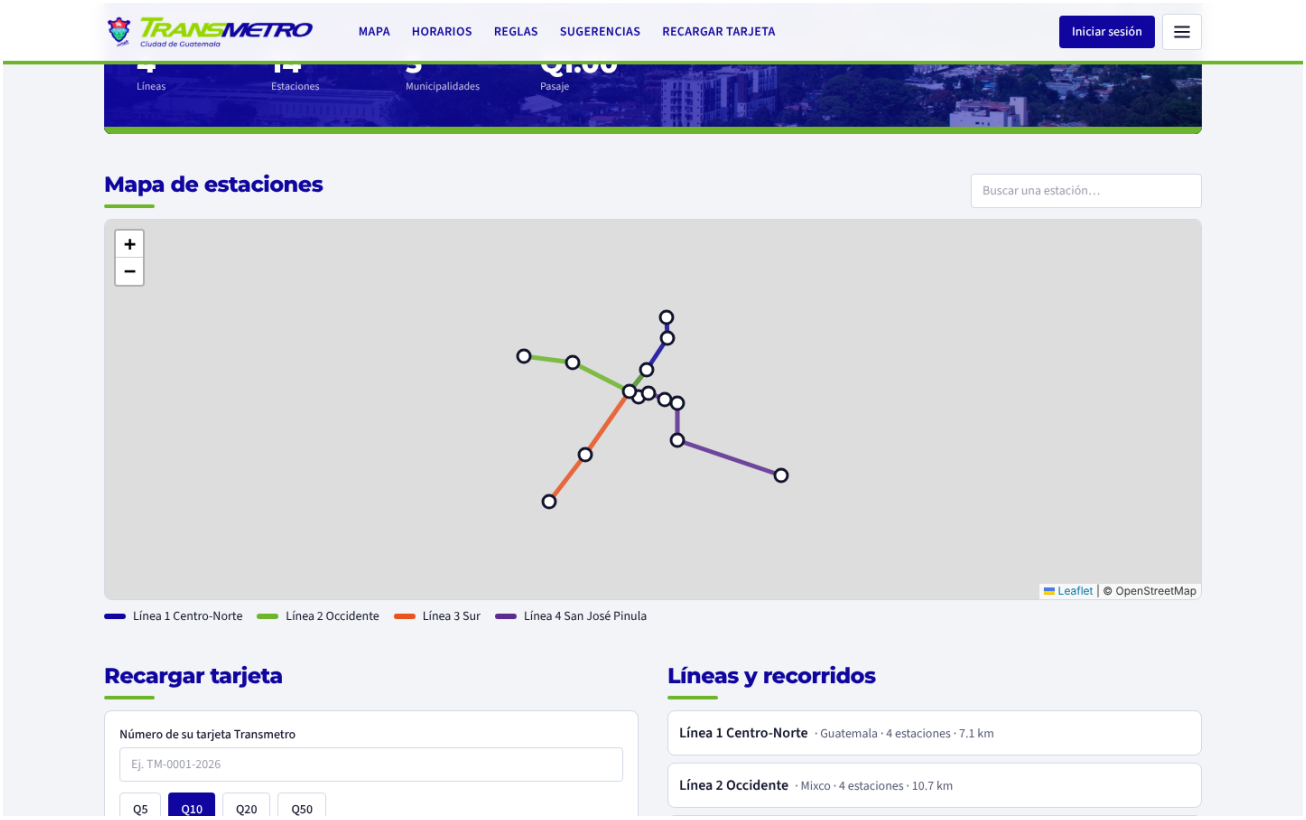
Evidencias del sistema en funcionamiento

Capturas de las pantallas principales: vista pública, tablero, mapa en vivo, alertas, líneas, buses, cobros, finanzas, demanda, reportes, horarios y portal del pasajero.

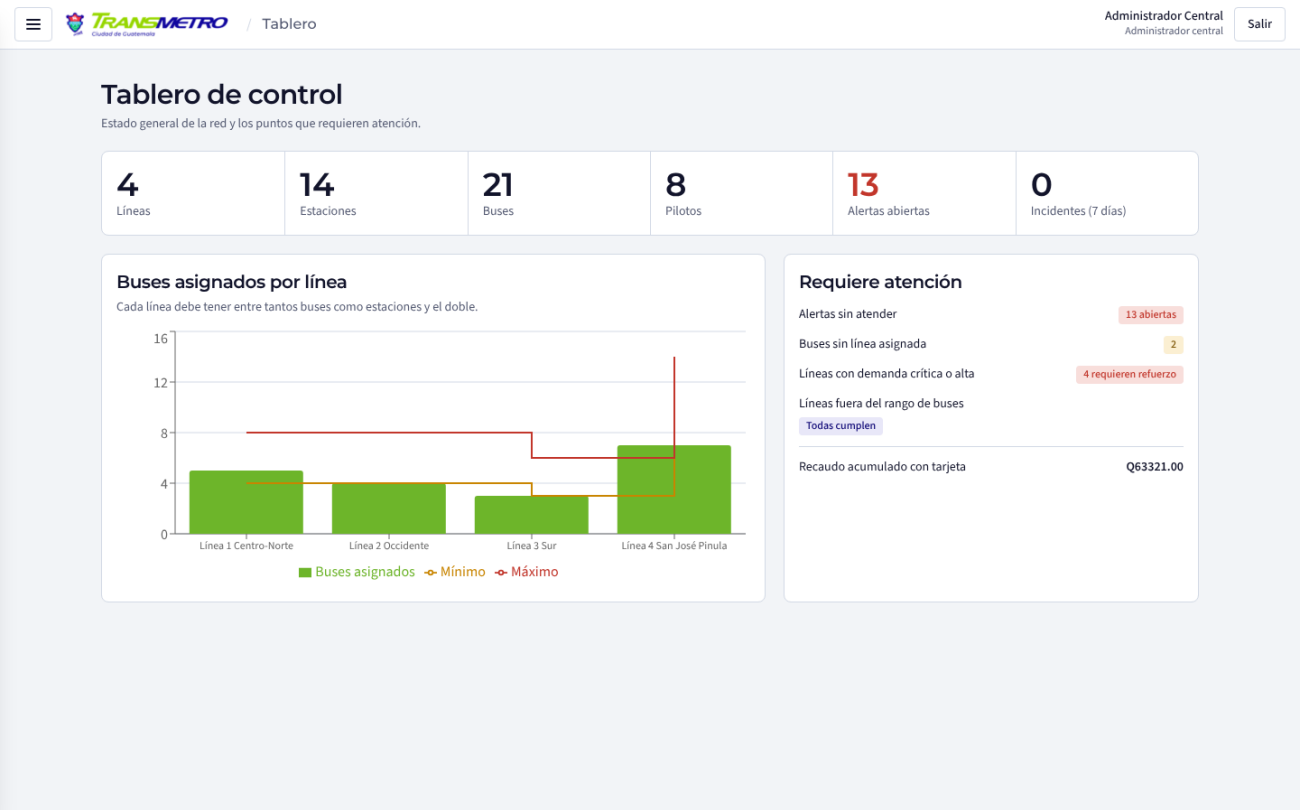
Capturas tomadas el 24-10-2026 con el sistema en ejecución, con los datos de demostración y el simulador de GPS activo. **Nota:** el entorno donde se tomaron no tenía acceso a los servidores de OpenStreetMap, por lo que los mapas muestran las líneas y estaciones sobre fondo gris; en el sitio publicado (https://transmetro-glvg.onrender.com) el mapa se muestra con el fondo de calles.



8.1 Pantalla principal pública: información general, cifras de la red y mapa de estaciones (sin iniciar sesión).



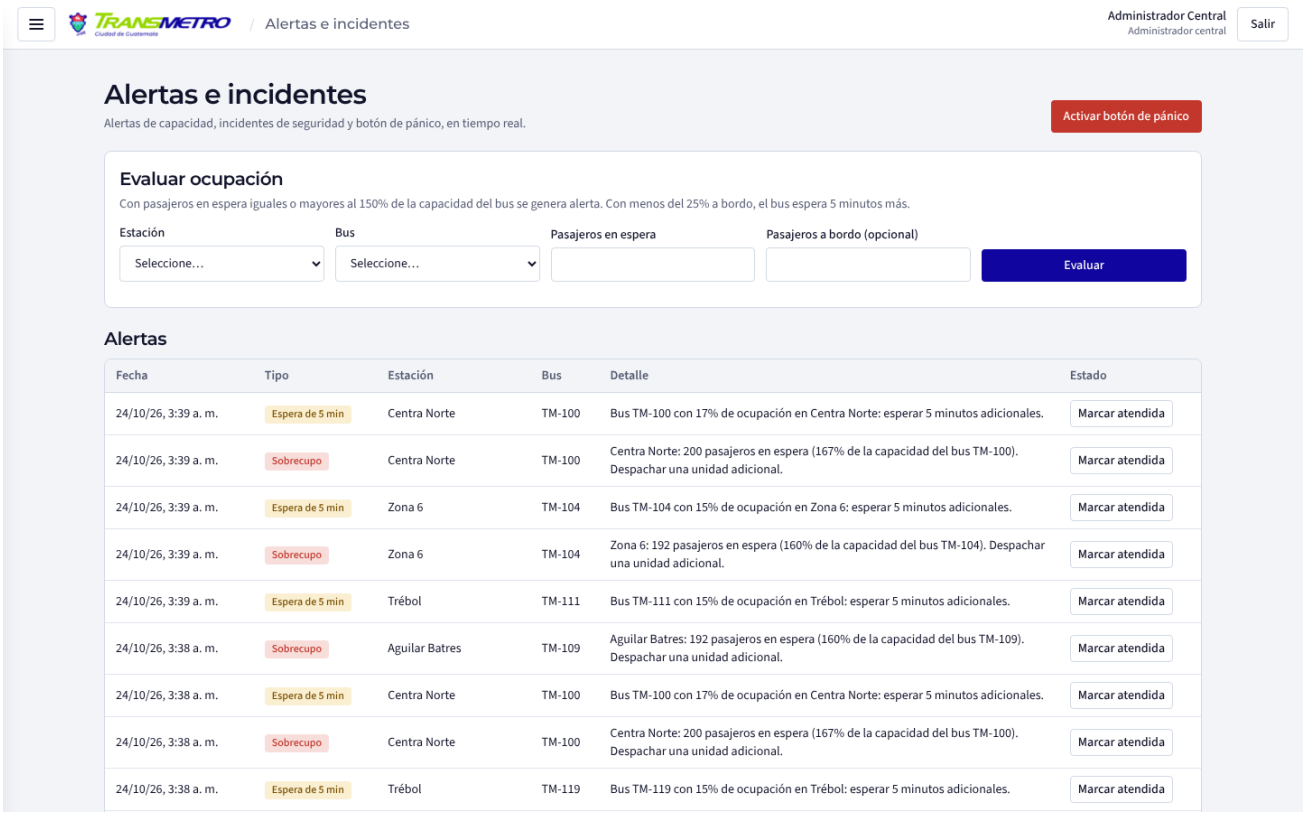
8.2 Mapa de estaciones con buscador, recarga de tarjeta sin sesión y recorridos de las líneas.



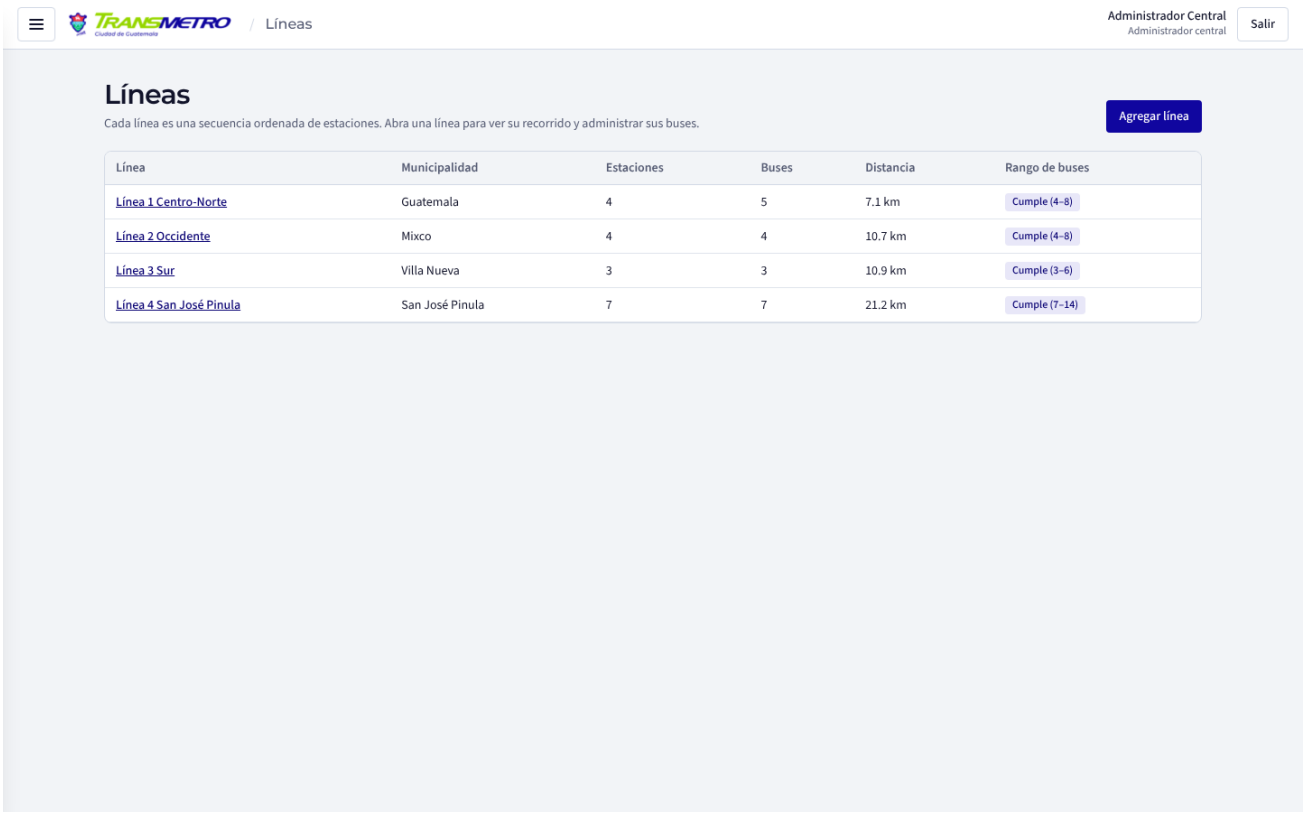
8.3 Tablero de control (RF-11): buses por línea frente al mínimo y máximo, alertas abiertas y puntos que requieren atención.



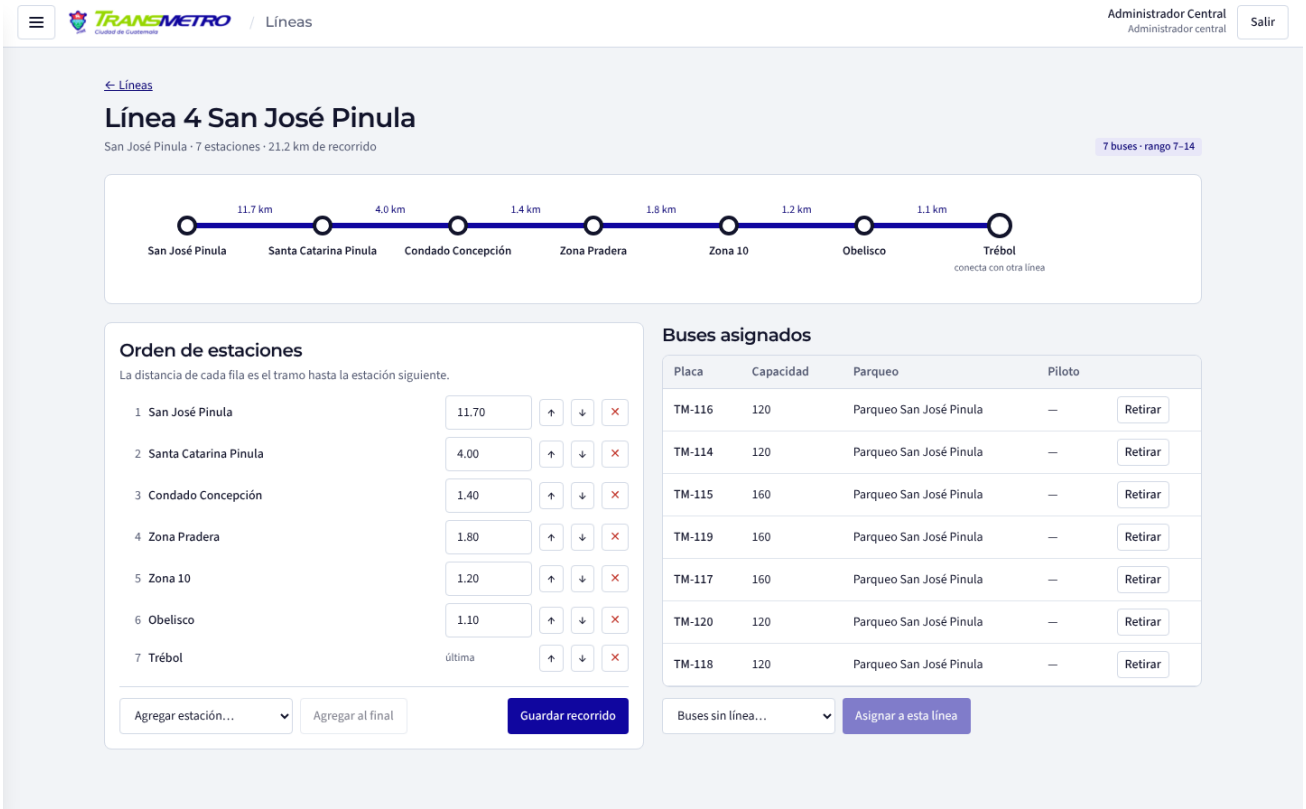
8.4 Mapa en vivo (RF-5): recorridos de las cuatro líneas y buses moviéndose en tiempo real; arriba, "Conectado en tiempo real".



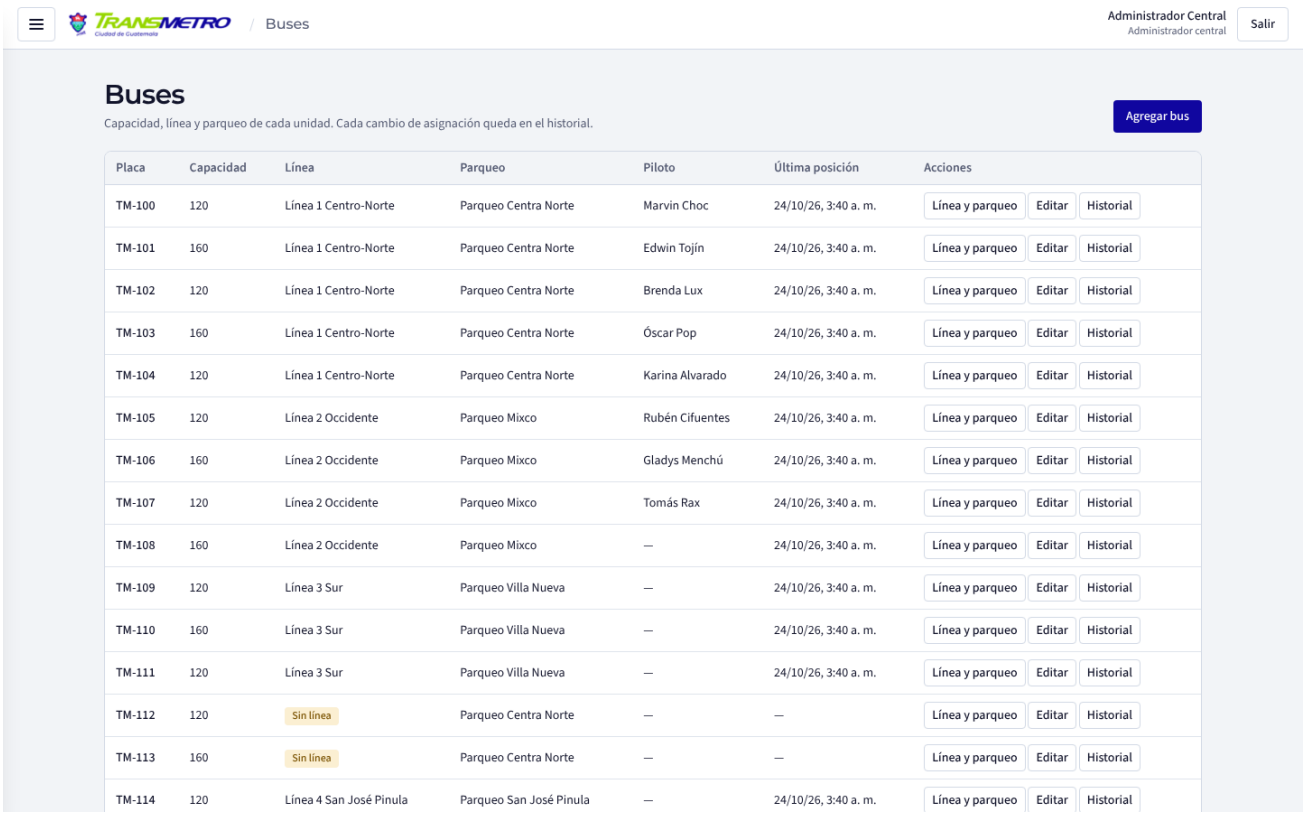
8.5 Alertas e incidentes (RF-7, RF-8): evaluación de ocupación, alertas de sobrecupo y de espera extra, y botón de pánico.



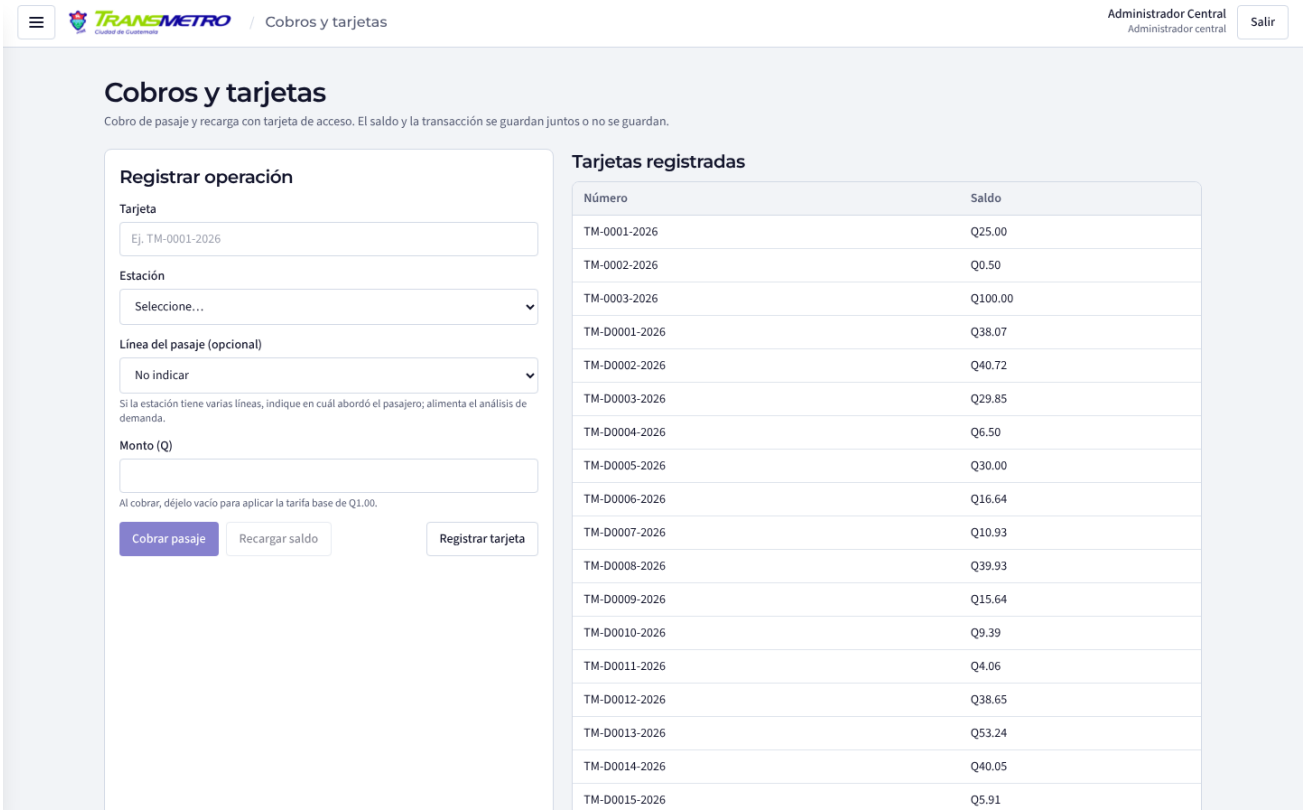
8.6 Líneas (RF-1, RF-3): estaciones, buses, distancia y cumplimiento del rango N a 2N.



8.7 Detalle de la Línea 4: recorrido ordenado con la distancia de cada tramo y buses asignados (7 de 7 a 14).



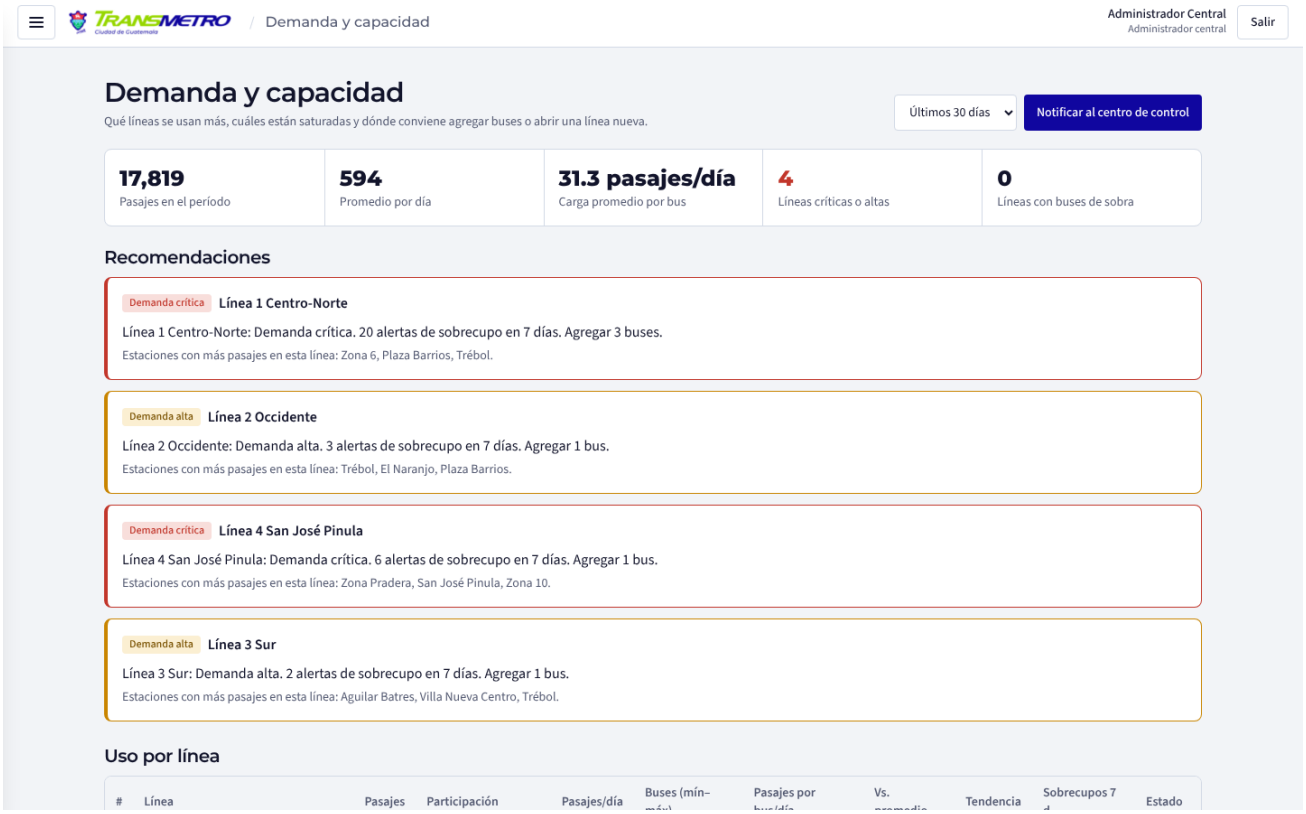
8.8 Buses (RF-3, RF-4, RNF-6): capacidad, línea, parqueo, piloto, última posición e historial.



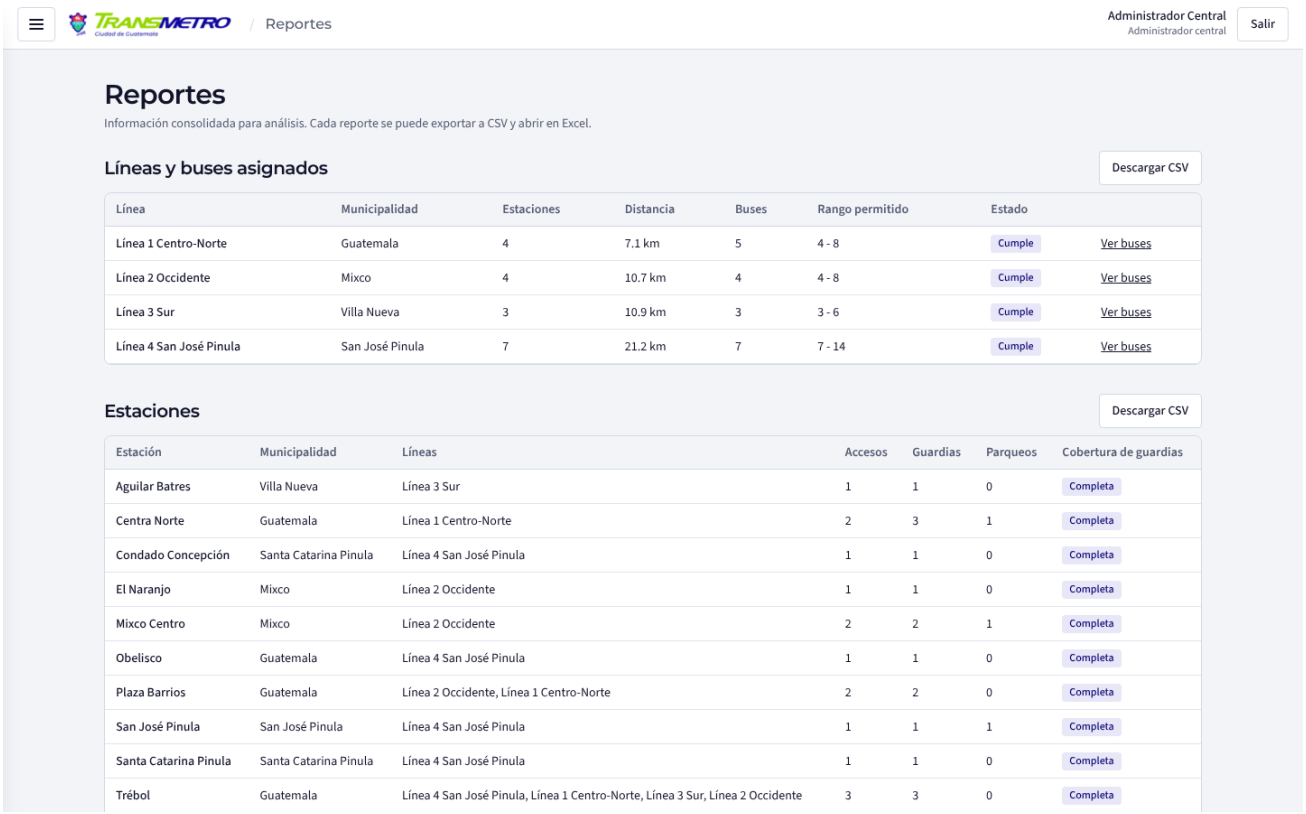
8.9 Cobros y tarjetas (RF-9): cobro de pasaje y recarga con tarjeta de acceso.



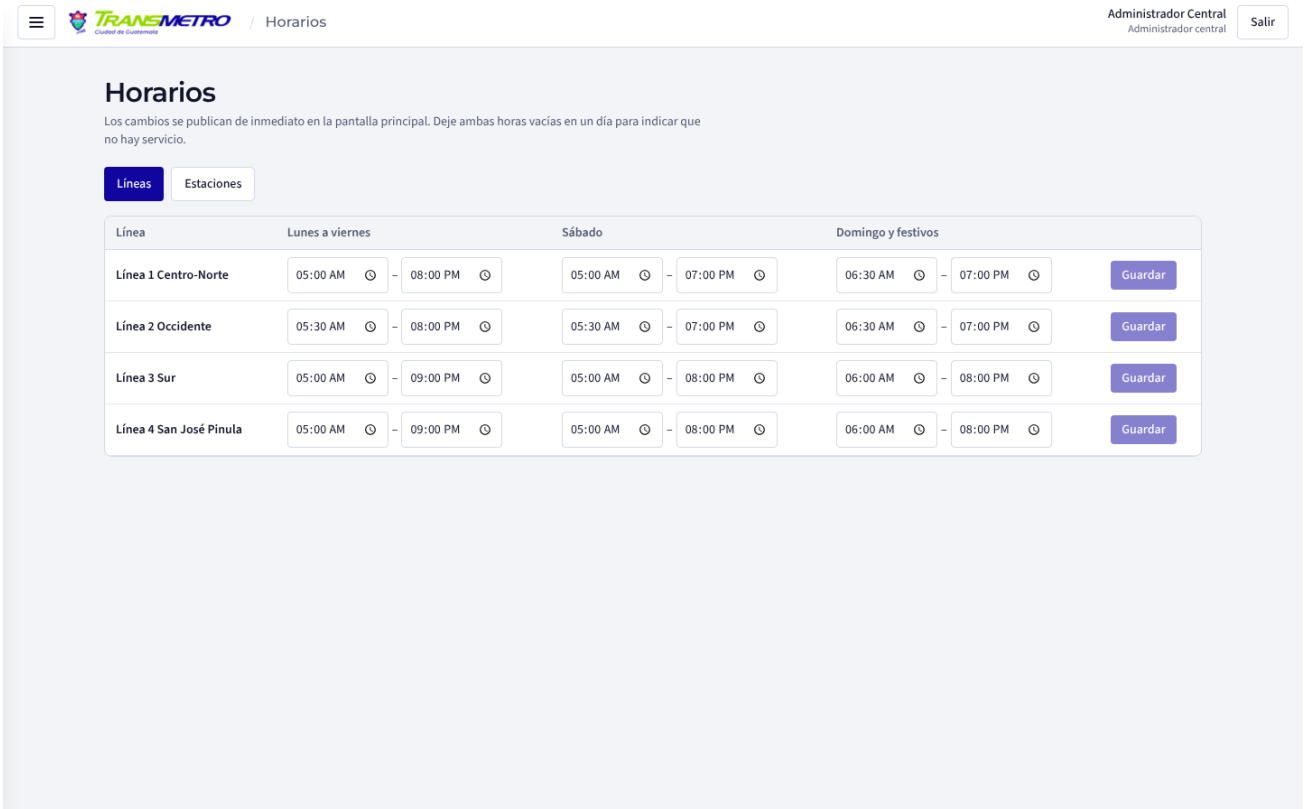
8.10 Finanzas: flujo de caja, cobertura de costos y costo por pasajero (datos de demostración).



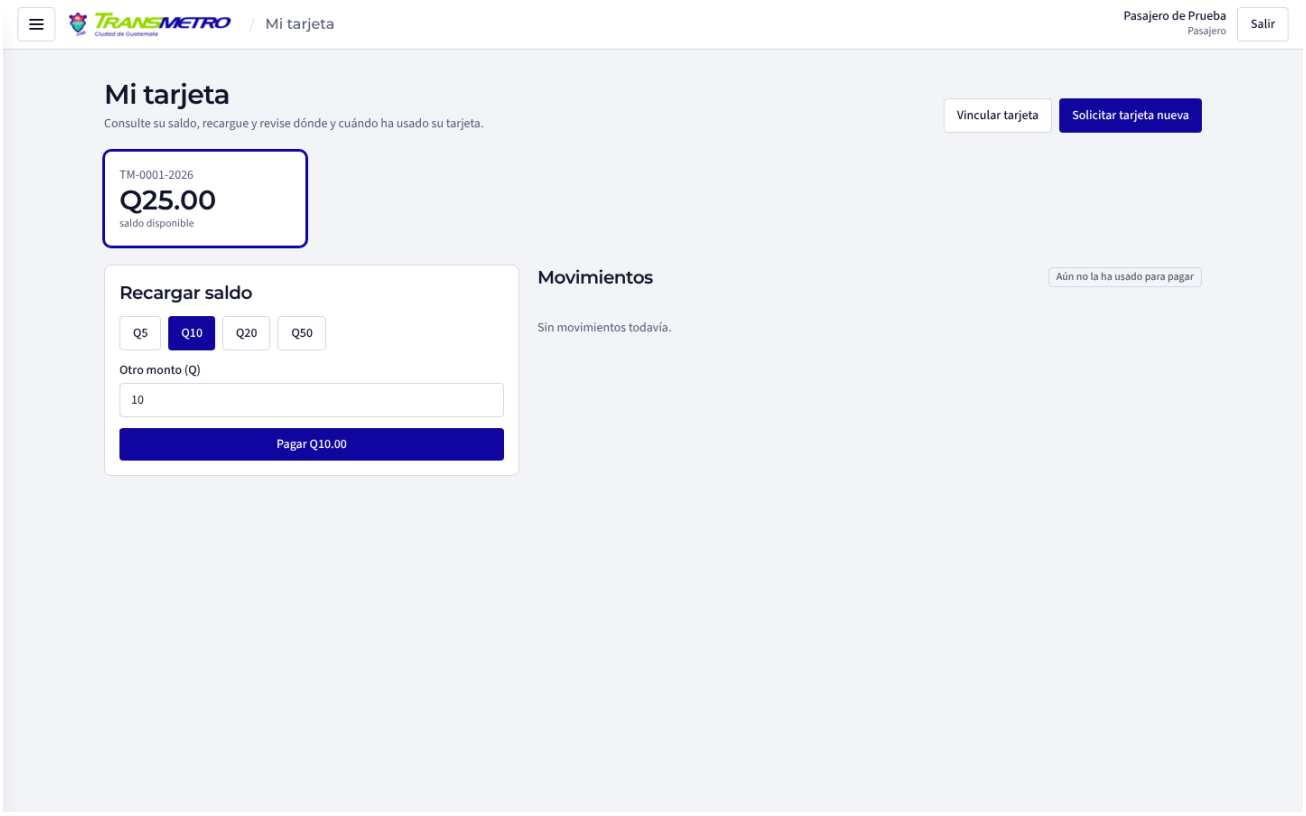
8.11 Demanda y capacidad: líneas críticas y recomendaciones de refuerzo de flota.



8.12 Reportes (RF-10): líneas con buses asignados y estaciones con cobertura de guardias, exportables a CSV.



8.13 Horarios de servicio editables por el administrador.



8.14 Portal del pasajero: saldo, recarga en línea y movimientos de su tarjeta.

9

Conclusiones y enlaces del proyecto

Conclusiones, trabajo pendiente, enlaces al repositorio y a la aplicación publicada, cuentas de demostración y ejecución local.

9. Conclusiones

- El sistema implementa las reglas de negocio centrales del enunciado (rango de N a 2N buses por línea, bus siempre con parqueo, ocupación con 150 % y 25 %, cobro atómico e idempotente) en la capa de servicios del backend, de modo que se cumplen sin importar desde dónde llegue la petición.
- El plan de pruebas es trazable a cada requerimiento: 55 de los 60 casos se ejecutaron, con una tasa de éxito de 98.2 %, y las 103 pruebas automatizadas aprueban.
- La cobertura de código del backend (93.7 % de líneas y 98 % a 100 % en los servicios críticos) supera las metas definidas en el stack tecnológico.
- Las mediciones locales de tiempo real (p95 de 7 ms) y de respuesta de la API (p95 de 284 ms) están dentro de las metas de RNF-2.
- El ciclo de pruebas encontró 8 defectos; 6 están corregidos y 2 abiertos (dependencias con vulnerabilidades que requieren actualizaciones mayores). La densidad de defectos (1.5 por KLOC) no alcanza la meta de menos de 1 por KLOC.
- La aplicación está publicada en la nube y el código, los scripts de base de datos y la documentación están en un repositorio público.

Trabajo pendiente

Pendiente	Requerimiento	Siguiente paso
Operación sin conexión de la PC de estación	RF-12	PWA con service worker e IndexedDB, sincronizando con la referencia idempotente ya existente
Integración con cámaras IP, validadores y GPS físicos	RF-5, RF-8, RF-9	Conectar los dispositivos a los endpoints actuales; la pasarela de pago está detrás de un adaptador
Pruebas de extremo a extremo y de carga	RNF-2, RNF-4	Playwright para flujos de navegador y k6 contra el sitio publicado (CP-52 a CP-54)
Métricas de código e integración continua	Calidad	SonarQube y GitHub Actions en cada cambio
Documentación OpenAPI	RNF-5	Swagger a partir de los esquemas de Zod (CP-58)
Migraciones versionadas y respaldo programado	RNF-7	Migraciones de Sequelize y respaldos automáticos (pg_dump programado o los de Neon)
Vulnerabilidades abiertas	RNF-3	Migrar a zod 4 y actualizar react-router y Sequelize (CP-57)
Cifrado de datos de pilotos	RNF-3	Aplicar el cifrado AES-256-GCM ya usado para DPI y NIT a los datos personales de pilotos

10. Enlaces y acceso al proyecto

Repositorio: https://github.com/ernesto615/Transmetro

Aplicación publicada: https://transmetro-glvg.onrender.com

Video de explicación general: [Agregar aquí el enlace del video]

El plan gratuito de Render suspende el servicio después de 15 minutos sin visitas; la primera visita puede tardar cerca de un minuto en responder.

Verificación rápida

Dirección	Resultado esperado
https://transmetro-glvg.onrender.com/health	Responde {"ok": true}
https://transmetro-glvg.onrender.com/api/publico/red	JSON con las líneas, estaciones y horarios de la red
https://transmetro-glvg.onrender.com	Pantalla principal pública con el mapa

Cuentas de demostración

Rol	Correo	Contraseña
Administrador	admin@transmetro.gt	Se entrega por separado (variable SEED_ADMIN_PASSWORD)
Personal municipal	municipal@transmetro.gt	Muni12345!
Operador de estación	operador@transmetro.gt	Oper12345!
Guardia	guardia@transmetro.gt	Guardia123!
Pasajero	pasajero@transmetro.gt	Pasajero123!

Contenido del repositorio

Ruta	Contenido
README.md	Descripción, puesta en marcha local, reglas de negocio, endpoints y pruebas
DESPLIEGUE.md	Publicación paso a paso en Render y Neon
database/	Scripts de PostgreSQL: esquema, triggers, datos de demostración y consultas de reportes
backend/tests/	Pruebas unitarias y de integración (npm test)
pruebas/	Colección de Postman de la API
docs/	Guía técnica y guion de la presentación

Ejecución local

```
docker compose up -d                # PostgreSQL 16 y pgAdmin
cd backend && npm install && npm run seed && npm run dev   # API en http://localhost:4000
cd frontend && npm install && npm run dev                  # Web en http://localhost:5173
cd backend && npm test              # pruebas automáticas (base transmetro_test)
cd backend && npm run simular       # buses en movimiento y alertas
```

Anexo A. Script de la base de datos en PostgreSQL

Versión en PostgreSQL del script de la sección 2, con las tablas y restricciones agregadas. Archivo `database/01_esquema.sql` del repositorio.

```
-- ========================================================================
-- TRANSMETRO · Esquema PostgreSQL (equivale al que genera Sequelize)
-- Orden de ejecución: 01_esquema -> 02_reglas_triggers -> 03_datos_demo
-- ATENCIÓN: los DROP borran todo. Úselo solo en la base de desarrollo.
-- ========================================================================
DROP TABLE IF EXISTS movimiento_financiero, horario_estacion, horario_linea, sugerencia, auditoria_bus,
      usuario, incidente_seguridad, alerta, transaccion, tarjeta_acceso,
  piloto, bus, parqueo, operador, guardia, acceso, linea_estacion, estacion, linea, municipalidad CASCADE;

CREATE TABLE municipalidad (
  id_municipalidad SERIAL PRIMARY KEY,
  nombre           VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE linea (
  id_linea           SERIAL PRIMARY KEY,
  nombre             VARCHAR(100) NOT NULL,
  distancia_total_km NUMERIC(8,2) NOT NULL DEFAULT 0,      -- la mantiene un trigger
  id_municipalidad   INT NOT NULL REFERENCES municipalidad(id_municipalidad)
);

CREATE TABLE estacion (
  id_estacion      SERIAL PRIMARY KEY,
  nombre           VARCHAR(100) NOT NULL,
  id_municipalidad INT NOT NULL REFERENCES municipalidad(id_municipalidad),
  latitud          NUMERIC(9,6),
  longitud         NUMERIC(9,6)
);

-- Tabla intermedia: una estación puede estar en varias líneas y el orden va en la relación
CREATE TABLE linea_estacion (
  id_linea                 INT NOT NULL REFERENCES linea(id_linea),
  id_estacion              INT NOT NULL REFERENCES estacion(id_estacion),
  orden                    INT NOT NULL CHECK (orden > 0),
  distancia_a_siguiente_km NUMERIC(6,2) NOT NULL DEFAULT 0 CHECK (distancia_a_siguiente_km >= 0),
  PRIMARY KEY (id_linea, id_estacion),
  UNIQUE (id_linea, orden)
);

CREATE TABLE acceso (
  id_acceso   SERIAL PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  id_estacion INT NOT NULL REFERENCES estacion(id_estacion)
);

CREATE TABLE guardia (
  id_guardia SERIAL PRIMARY KEY,
  nombre     VARCHAR(100) NOT NULL,
  turno      VARCHAR(20),
  id_acceso  INT NOT NULL REFERENCES acceso(id_acceso)
);

CREATE TABLE operador (
  id_operador SERIAL PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  usuario     VARCHAR(50) NOT NULL UNIQUE,
  id_estacion INT NOT NULL REFERENCES estacion(id_estacion)
);

CREATE TABLE parqueo (
  id_parqueo  SERIAL PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  capacidad   INT CHECK (capacidad >= 0),
  id_estacion INT REFERENCES estacion(id_estacion)         -- "algunas estaciones tienen parqueo"
);
```

```
CREATE TABLE bus (
  id_bus           SERIAL PRIMARY KEY,
  placa            VARCHAR(15) NOT NULL UNIQUE,
  capacidad_maxima INT NOT NULL CHECK (capacidad_maxima > 0),
  id_linea         INT REFERENCES linea(id_linea),                    -- puede ser NULL (sin línea)
  id_parqueo       INT NOT NULL REFERENCES parqueo(id_parqueo),       -- NUNCA NULL (req. 7)
  latitud          NUMERIC(9,6),
  longitud         NUMERIC(9,6),
  ultima_posicion  TIMESTAMPTZ
);

CREATE TABLE piloto (
  id_piloto            SERIAL PRIMARY KEY,
  nombre               VARCHAR(100) NOT NULL,
  historial_educativo  VARCHAR(255),
  direccion_residencia VARCHAR(255),
  telefono             VARCHAR(20),
  email                VARCHAR(100),
  id_bus               INT REFERENCES bus(id_bus)
);

CREATE TABLE tarjeta_acceso (
  id_tarjeta SERIAL PRIMARY KEY,
  numero     VARCHAR(30) NOT NULL UNIQUE,
  saldo      NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (saldo >= 0),  -- el saldo nunca queda negativo
  id_usuario INT                                                  -- pasajero dueño (se agrega la FK al
      final)
);

CREATE TABLE transaccion (
  id_transaccion SERIAL PRIMARY KEY,
  id_tarjeta     INT NOT NULL REFERENCES tarjeta_acceso(id_tarjeta),
  id_estacion    INT REFERENCES estacion(id_estacion),               -- NULL en recargas en línea
  id_linea       INT REFERENCES linea(id_linea),                     -- línea del pasaje (análisis de
      demanda)
  fecha_hora     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  monto          NUMERIC(8,2) NOT NULL CHECK (monto > 0),
  medio_pago     VARCHAR(30) NOT NULL,                               -- TARJETA | RECARGA |
      RECARGA_EN_LINEA
  referencia     VARCHAR(64) UNIQUE                                  -- idempotencia: evita cobros
      duplicados
);

CREATE TABLE alerta (
  id_alerta   SERIAL PRIMARY KEY,
  tipo        VARCHAR(30) NOT NULL,                                  -- SOBRECUPO | ESPERA_EXTRA
  id_estacion INT REFERENCES estacion(id_estacion),
  id_bus      INT REFERENCES bus(id_bus),
  fecha_hora  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  descripcion VARCHAR(255),
  atendida    BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE incidente_seguridad (
  id_incidente SERIAL PRIMARY KEY,
  id_estacion  INT REFERENCES estacion(id_estacion),
  id_acceso    INT REFERENCES acceso(id_acceso),
  id_bus       INT REFERENCES bus(id_bus),
  fecha_hora   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  tipo         VARCHAR(50) NOT NULL,
  descripcion  VARCHAR(255)
);

CREATE TABLE usuario (
  id_usuario       SERIAL PRIMARY KEY,
  nombre           VARCHAR(100) NOT NULL,
  email            VARCHAR(120) NOT NULL UNIQUE,
  password_hash    VARCHAR(100) NOT NULL,
  rol              VARCHAR(20) NOT NULL CHECK (rol IN ('admin','municipal','operador','guardia',
      'pasajero')),
  id_estacion      INT REFERENCES estacion(id_estacion),
  id_municipalidad INT REFERENCES municipalidad(id_municipalidad),
```

```
  activo           BOOLEAN NOT NULL DEFAULT TRUE,
  telefono         VARCHAR(20),
  dpi_cifrado      VARCHAR(200),               -- AES-256-GCM; lo cifra la API
  dpi_hash         VARCHAR(64) UNIQUE,         -- huella HMAC para detectar duplicados
  nit_cifrado      VARCHAR(200),
  nit_hash         VARCHAR(64) UNIQUE
);

CREATE TABLE auditoria_bus (
  id_auditoria        SERIAL PRIMARY KEY,
  id_bus              INT NOT NULL REFERENCES bus(id_bus),
  id_linea_anterior   INT,
  id_linea_nueva      INT,
  id_parqueo_anterior INT,
  id_parqueo_nuevo    INT,
  id_usuario          INT,
  fecha_hora          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

ALTER TABLE tarjeta_acceso ADD CONSTRAINT fk_tarjeta_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(
      id_usuario);

-- Libro de ingresos y egresos (dashboard financiero del administrador)
CREATE TABLE movimiento_financiero (
  id_movimiento SERIAL PRIMARY KEY,
  tipo          VARCHAR(10) NOT NULL CHECK (tipo IN ('INGRESO','EGRESO')),
  categoria     VARCHAR(60) NOT NULL,
  monto         NUMERIC(12,2) NOT NULL CHECK (monto > 0),
  fecha         DATE NOT NULL,
  descripcion   VARCHAR(255),
  id_linea      INT REFERENCES linea(id_linea),
  id_usuario    INT REFERENCES usuario(id_usuario),
  creado        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Caja de sugerencias y quejas (formulario público)
CREATE TABLE sugerencia (
  id_sugerencia SERIAL PRIMARY KEY,
  tipo          VARCHAR(20) NOT NULL CHECK (tipo IN ('SUGERENCIA','QUEJA')),
  nombre        VARCHAR(100),
  contacto      VARCHAR(120),
  id_linea      INT REFERENCES linea(id_linea),
  id_estacion   INT REFERENCES estacion(id_estacion),
  mensaje       TEXT NOT NULL,
  fecha_hora    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  estado        VARCHAR(20) NOT NULL DEFAULT 'NUEVA' CHECK (estado IN ('NUEVA','EN_REVISION','RESUELTA')),
  nota_interna  VARCHAR(500)
);

-- Horarios editables por el administrador; sin fila = sin servicio ese día
CREATE TABLE horario_linea (
  id_linea    INT NOT NULL REFERENCES linea(id_linea) ON DELETE CASCADE,
  tipo_dia    VARCHAR(20) NOT NULL CHECK (tipo_dia IN ('LUNES_VIERNES','SABADO','DOMINGO_FESTIVOS')),
  hora_inicio VARCHAR(5) NOT NULL,
  hora_fin    VARCHAR(5) NOT NULL,
  PRIMARY KEY (id_linea, tipo_dia),
  CHECK (hora_inicio < hora_fin)
);
CREATE TABLE horario_estacion (
  id_estacion INT NOT NULL REFERENCES estacion(id_estacion) ON DELETE CASCADE,
  tipo_dia    VARCHAR(20) NOT NULL CHECK (tipo_dia IN ('LUNES_VIERNES','SABADO','DOMINGO_FESTIVOS')),
  hora_inicio VARCHAR(5) NOT NULL,
  hora_fin    VARCHAR(5) NOT NULL,
  PRIMARY KEY (id_estacion, tipo_dia),
  CHECK (hora_inicio < hora_fin)
);

CREATE INDEX idx_bus_linea     ON bus(id_linea);
CREATE INDEX idx_acceso_est    ON acceso(id_estacion);
CREATE INDEX idx_alerta_estado ON alerta(atendida, fecha_hora DESC);
CREATE INDEX idx_tx_fecha      ON transaccion(fecha_hora DESC);
CREATE INDEX idx_tx_linea      ON transaccion(id_linea, fecha_hora);
```

```
CREATE INDEX idx_mov_fecha     ON movimiento_financiero(fecha);
```

Anexo A.2 Reglas de negocio en la base de datos (triggers)

Segunda capa de validación del rango de buses por línea y de la distancia total. Archivo `database/02_reglas_triggers.sql` del repositorio.

```
-- ======================================================================
-- Reglas de negocio aplicadas EN la base de datos (segunda línea de defensa).
-- La API ya las valida en services/, pero así nadie puede saltárselas con un INSERT manual.
-- Ejecutar DESPUÉS de 01_esquema.sql y ANTES de cargar datos.
-- ======================================================================

-- Regla: una línea admite entre N y 2N buses, siendo N su número de estaciones.

-- (a) Al asignar un bus a una línea: no superar el máximo (2N)
CREATE OR REPLACE FUNCTION fn_bus_validar_linea() RETURNS trigger AS $$
DECLARE n INT; actuales INT;
BEGIN
  IF NEW.id_linea IS NULL THEN RETURN NEW; END IF;
  IF TG_OP = 'UPDATE' THEN
    IF NEW.id_linea IS NOT DISTINCT FROM OLD.id_linea THEN RETURN NEW; END IF;
  END IF;
  SELECT COUNT(*) INTO n FROM linea_estacion WHERE id_linea = NEW.id_linea;
  IF n = 0 THEN
    RAISE EXCEPTION 'La línea % no tiene estaciones; defina su recorrido antes de asignarle buses', NEW.
      id_linea;
  END IF;
  SELECT COUNT(*) INTO actuales FROM bus WHERE id_linea = NEW.id_linea AND id_bus <> COALESCE(NEW.id_bus,
      -1);
  IF actuales + 1 > n * 2 THEN
    RAISE EXCEPTION 'La línea % ya tiene el máximo de buses permitido (% = 2 x % estaciones)', NEW.
      id_linea, n * 2, n;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_bus_validar_linea ON bus;
CREATE TRIGGER trg_bus_validar_linea BEFORE INSERT OR UPDATE OF id_linea ON bus
  FOR EACH ROW EXECUTE FUNCTION fn_bus_validar_linea();

-- (b) Al sacar un bus de una línea (cambio de línea o borrado): no bajar del mínimo (N)
CREATE OR REPLACE FUNCTION fn_validar_salida_linea(p_linea INT) RETURNS void AS $$
DECLARE n INT; actuales INT;
BEGIN
  IF p_linea IS NULL THEN RETURN; END IF;
  SELECT COUNT(*) INTO n FROM linea_estacion WHERE id_linea = p_linea;
  SELECT COUNT(*) INTO actuales FROM bus WHERE id_linea = p_linea;
  IF n > 0 AND actuales = n THEN
    RAISE EXCEPTION 'La línea % quedaría con menos buses que estaciones (mínimo %)', p_linea, n;
  END IF;
END $$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION fn_bus_salida_update() RETURNS trigger AS $$
BEGIN
  IF OLD.id_linea IS NOT NULL AND NEW.id_linea IS DISTINCT FROM OLD.id_linea THEN
    PERFORM fn_validar_salida_linea(OLD.id_linea);
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION fn_bus_salida_delete() RETURNS trigger AS $$
BEGIN
  PERFORM fn_validar_salida_linea(OLD.id_linea);
  RETURN OLD;
END $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_bus_salida_update ON bus;
CREATE TRIGGER trg_bus_salida_update BEFORE UPDATE OF id_linea ON bus
  FOR EACH ROW EXECUTE FUNCTION fn_bus_salida_update();
DROP TRIGGER IF EXISTS trg_bus_salida_delete ON bus;
CREATE TRIGGER trg_bus_salida_delete BEFORE DELETE ON bus
```

```
  FOR EACH ROW EXECUTE FUNCTION fn_bus_salida_delete();

-- Regla: la distancia total de la línea es la suma de los tramos entre estaciones.
CREATE OR REPLACE FUNCTION fn_actualizar_distancia_linea() RETURNS trigger AS $$
DECLARE l INT;
BEGIN
  IF TG_OP = 'DELETE' THEN l := OLD.id_linea; ELSE l := NEW.id_linea; END IF;
  UPDATE linea
     SET distancia_total_km = (SELECT COALESCE(SUM(distancia_a_siguiente_km), 0) FROM linea_estacion
      WHERE id_linea = l)
   WHERE id_linea = l;
  RETURN NULL;
END $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_distancia_linea ON linea_estacion;
CREATE TRIGGER trg_distancia_linea AFTER INSERT OR UPDATE OR DELETE ON linea_estacion
  FOR EACH ROW EXECUTE FUNCTION fn_actualizar_distancia_linea();

-- Reglas que ya cubre el propio esquema (01_esquema.sql):
--   * bus.id_parqueo NOT NULL            -> un bus nunca queda sin parqueo
--   * bus.id_linea es una sola columna   -> un bus pertenece a una sola línea
--   * tarjeta_acceso.saldo CHECK >= 0    -> no hay saldo negativo
--   * transaccion.referencia UNIQUE      -> un reintento no cobra dos veces
```

Anexo B. Consultas de los reportes

Constancia de los queries: cada consulta corresponde a un endpoint de la API y a una pantalla de la aplicación. Archivo `database/04_consultas_reportes.sql` del repositorio.

```
-- ======================================================================
-- Consultas de los reportes (constancia de los queries para el documento).
-- Cada una corresponde a un endpoint de la API y a una pantalla de la web.
-- ======================================================================

-- Q1 · Reporte de estaciones           (GET /api/reportes/estaciones · pantalla Reportes)
CREATE OR REPLACE VIEW v_reporte_estaciones AS
SELECT e.id_estacion,
       e.nombre                                        AS estacion,
       m.nombre                                        AS municipalidad,
       (SELECT STRING_AGG(l.nombre, ', ' ORDER BY l.nombre)
          FROM linea_estacion le JOIN linea l ON l.id_linea = le.id_linea
         WHERE le.id_estacion = e.id_estacion)         AS lineas,
       (SELECT COUNT(*) FROM acceso a WHERE a.id_estacion = e.id_estacion) AS accesos,
       (SELECT COUNT(*) FROM guardia g JOIN acceso a ON a.id_acceso = g.id_acceso
         WHERE a.id_estacion = e.id_estacion)          AS guardias,
       (SELECT COUNT(*) FROM parqueo p WHERE p.id_estacion = e.id_estacion) AS parqueos,
       -- cumple si tiene accesos y ninguno está sin guardia
       (EXISTS (SELECT 1 FROM acceso a WHERE a.id_estacion = e.id_estacion)
        AND NOT EXISTS (SELECT 1 FROM acceso a
                        WHERE a.id_estacion = e.id_estacion
                          AND NOT EXISTS (SELECT 1 FROM guardia g WHERE g.id_acceso = a.id_acceso))) AS
     cumple_guardia_minima
FROM estacion e
JOIN municipalidad m ON m.id_municipalidad = e.id_municipalidad;

SELECT * FROM v_reporte_estaciones ORDER BY estacion;

-- Q2 · Líneas con buses asignados y verificación del rango N..2N   (GET /api/reportes/lineas-buses)
CREATE OR REPLACE VIEW v_reporte_lineas_buses AS
WITH n AS (SELECT id_linea, COUNT(*) AS estaciones FROM linea_estacion GROUP BY id_linea),
     b AS (SELECT id_linea, COUNT(*) AS buses FROM bus WHERE id_linea IS NOT NULL GROUP BY id_linea)
SELECT l.id_linea,
       l.nombre                          AS linea,
       m.nombre                          AS municipalidad,
       l.distancia_total_km,
       COALESCE(n.estaciones, 0)         AS estaciones,
       COALESCE(b.buses, 0)              AS buses_asignados,
       COALESCE(n.estaciones, 0) || ' - ' || COALESCE(n.estaciones, 0) * 2 AS rango_permitido,
       COALESCE(n.estaciones, 0) > 0
         AND COALESCE(b.buses, 0) BETWEEN COALESCE(n.estaciones, 0) AND COALESCE(n.estaciones, 0) * 2 AS
     cumple_rango
FROM linea l
JOIN municipalidad m ON m.id_municipalidad = l.id_municipalidad
LEFT JOIN n ON n.id_linea = l.id_linea
LEFT JOIN b ON b.id_linea = l.id_linea;

SELECT * FROM v_reporte_lineas_buses ORDER BY linea;

-- Q3 · Todos los accesos de cada línea  (GET /api/reportes/accesos-por-linea)
SELECT l.nombre AS linea, le.orden, e.nombre AS estacion, a.nombre AS acceso
FROM linea l
JOIN linea_estacion le ON le.id_linea = l.id_linea
JOIN estacion e        ON e.id_estacion = le.id_estacion
JOIN acceso a          ON a.id_estacion = e.id_estacion
ORDER BY l.nombre, le.orden, a.nombre;

-- Q4 · Detalle de buses de una línea, con parqueo y piloto
SELECT b.placa, b.capacidad_maxima, p.nombre AS parqueo, pi.nombre AS piloto
FROM bus b
JOIN parqueo p     ON p.id_parqueo = b.id_parqueo
LEFT JOIN piloto pi ON pi.id_bus = b.id_bus
WHERE b.id_linea = (SELECT id_linea FROM linea WHERE nombre = 'Línea 1 Centro-Norte')
ORDER BY b.placa;
-- Q5 · Distancia entre dos estaciones de una misma línea (suma de tramos)
```

```
SELECT SUM(le.distancia_a_siguiente_km) AS km
FROM linea_estacion le
WHERE le.id_linea = (SELECT id_linea FROM linea WHERE nombre = 'Línea 1 Centro-Norte')
  AND le.orden >= (SELECT orden FROM linea_estacion WHERE id_linea = le.id_linea
                   AND id_estacion = (SELECT id_estacion FROM estacion WHERE nombre = 'Centra Norte'))
  AND le.orden <  (SELECT orden FROM linea_estacion WHERE id_linea = le.id_linea
                   AND id_estacion = (SELECT id_estacion FROM estacion WHERE nombre = 'Plaza Barrios'));

-- Q6 · Estaciones que pertenecen a más de una línea (relación muchos a muchos)
SELECT e.nombre AS estacion, COUNT(*) AS lineas
FROM linea_estacion le JOIN estacion e ON e.id_estacion = le.id_estacion
GROUP BY e.nombre HAVING COUNT(*) > 1;

-- Q7 · Buses sin línea asignada
SELECT placa, capacidad_maxima FROM bus WHERE id_linea IS NULL;

-- Q8 · Recaudo por estación (solo cobros de pasaje)
SELECT e.nombre AS estacion, COUNT(*) AS cobros, SUM(t.monto) AS total
FROM transaccion t JOIN estacion e ON e.id_estacion = t.id_estacion
WHERE t.medio_pago = 'TARJETA'
GROUP BY e.nombre ORDER BY total DESC;

-- Q9 · Alertas abiertas por tipo (tablero)
SELECT tipo, COUNT(*) AS abiertas FROM alerta WHERE NOT atendida GROUP BY tipo;

-- Q10 · Historial de cambios de asignación de un bus (RNF-6)
SELECT a.fecha_hora, a.id_linea_anterior, a.id_linea_nueva, a.id_parqueo_anterior, a.id_parqueo_nuevo
FROM auditoria_bus a JOIN bus b ON b.id_bus = a.id_bus
WHERE b.placa = 'TM-100' ORDER BY a.fecha_hora DESC;

-- Q11 · ¿Dónde y cuándo pagó o recargó una tarjeta? (GET /api/mi/tarjetas/:numero/historial)
SELECT t.fecha_hora, t.medio_pago, t.monto, COALESCE(e.nombre, 'En línea') AS lugar
FROM transaccion t
JOIN tarjeta_acceso c ON c.id_tarjeta = t.id_tarjeta
LEFT JOIN estacion e  ON e.id_estacion = t.id_estacion
WHERE c.numero = 'TM-0001-2026'
ORDER BY t.fecha_hora DESC;

-- Q12 · Último uso (pasaje) de cada tarjeta, con su dueño
SELECT c.numero, u.nombre AS pasajero, MAX(t.fecha_hora) AS ultimo_uso
FROM tarjeta_acceso c
LEFT JOIN usuario u ON u.id_usuario = c.id_usuario
LEFT JOIN transaccion t ON t.id_tarjeta = c.id_tarjeta AND t.medio_pago = 'TARJETA'
GROUP BY c.numero, u.nombre;

-- Q13 · Horarios de servicio por línea (se muestran en la pantalla principal)
SELECT l.nombre AS linea,
       MAX(CASE WHEN h.tipo_dia='LUNES_VIERNES'    THEN h.hora_inicio || ' a ' || h.hora_fin END) AS
      lunes_a_viernes,
       MAX(CASE WHEN h.tipo_dia='SABADO'           THEN h.hora_inicio || ' a ' || h.hora_fin END) AS
      sabado,
       MAX(CASE WHEN h.tipo_dia='DOMINGO_FESTIVOS' THEN h.hora_inicio || ' a ' || h.hora_fin END) AS
      domingo_y_festivos
FROM linea l LEFT JOIN horario_linea h ON h.id_linea = l.id_linea
GROUP BY l.nombre ORDER BY l.nombre;

-- Q14 · Sugerencias y quejas pendientes
SELECT fecha_hora, tipo, estado, COALESCE(nombre,'Anónimo') AS de, LEFT(mensaje, 80) AS mensaje
FROM sugerencia WHERE estado <> 'RESUELTA' ORDER BY fecha_hora DESC;

-- Q15 · Pasajeros con su documento registrado (los números están cifrados: solo se ve si existen)
SELECT nombre, email, telefono, dpi_hash IS NOT NULL AS tiene_dpi, nit_hash IS NOT NULL AS tiene_nit
FROM usuario WHERE rol = 'pasajero';

-- Q16 · Flujo de caja mensual: entradas (recargas + otros ingresos) menos salidas (egresos)
SELECT to_char(m.mes, 'YYYY-MM') AS mes,
       COALESCE(t.recargas, 0) + COALESCE(v.ingresos, 0) AS entradas,
       COALESCE(v.egresos, 0) AS salidas,
       COALESCE(t.recargas, 0) + COALESCE(v.ingresos, 0) - COALESCE(v.egresos, 0) AS flujo_neto
FROM generate_series(date_trunc('month', now()) - interval '5 months', date_trunc('month', now()),
      interval '1 month') AS m(mes)
LEFT JOIN (SELECT date_trunc('month', fecha_hora) AS mes, SUM(monto) AS recargas
```

```
           FROM transaccion WHERE medio_pago LIKE 'RECARGA%' GROUP BY 1) t ON t.mes = m.mes
LEFT JOIN (SELECT date_trunc('month', fecha::timestamp) AS mes,
                  SUM(monto) FILTER (WHERE tipo = 'INGRESO') AS ingresos, SUM(monto) FILTER (WHERE tipo =
      'EGRESO') AS egresos
           FROM movimiento_financiero GROUP BY 1) v ON v.mes = m.mes
ORDER BY m.mes;

-- Q17 · Línea más usada: pasajes en los últimos 30 días y pasajes por bus por día
SELECT l.nombre AS linea,
       COUNT(t.id_transaccion) AS pasajes,
       ROUND(COUNT(t.id_transaccion) / 30.0, 1) AS pasajes_por_dia,
       (SELECT COUNT(*) FROM bus b WHERE b.id_linea = l.id_linea) AS buses,
       ROUND(COUNT(t.id_transaccion) / 30.0 / NULLIF((SELECT COUNT(*) FROM bus b WHERE b.id_linea = l.
      id_linea), 0), 1) AS pasajes_por_bus_dia
FROM linea l
LEFT JOIN transaccion t ON t.id_linea = l.id_linea AND t.medio_pago = 'TARJETA' AND t.fecha_hora >= now()
      - interval '30 days'
GROUP BY l.id_linea, l.nombre ORDER BY pasajes DESC;

-- Q18 · Alertas de sobrecupo por línea en los últimos 7 días
SELECT l.nombre AS linea, COUNT(*) AS sobrecupos
FROM alerta a JOIN bus b ON b.id_bus = a.id_bus JOIN linea l ON l.id_linea = b.id_linea
WHERE a.tipo = 'SOBRECUPO' AND a.fecha_hora >= now() - interval '7 days'
GROUP BY l.nombre ORDER BY sobrecupos DESC;
```